TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM

Presenter Script – Full Session

Session	TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM
Date	Thursday, 29 October 2026
Location	SAP TechEd 2026, Berlin
Presenters	Christina Pickhardt · Niclas Weidner · Tobias Keller
Scenario	Sales Order Processing with Customer Down Payment (BKJ)

> This file is the compiled presenter script. Source of truth lives in the individual exercise READMEs under . Images are referenced relative to the repo root.

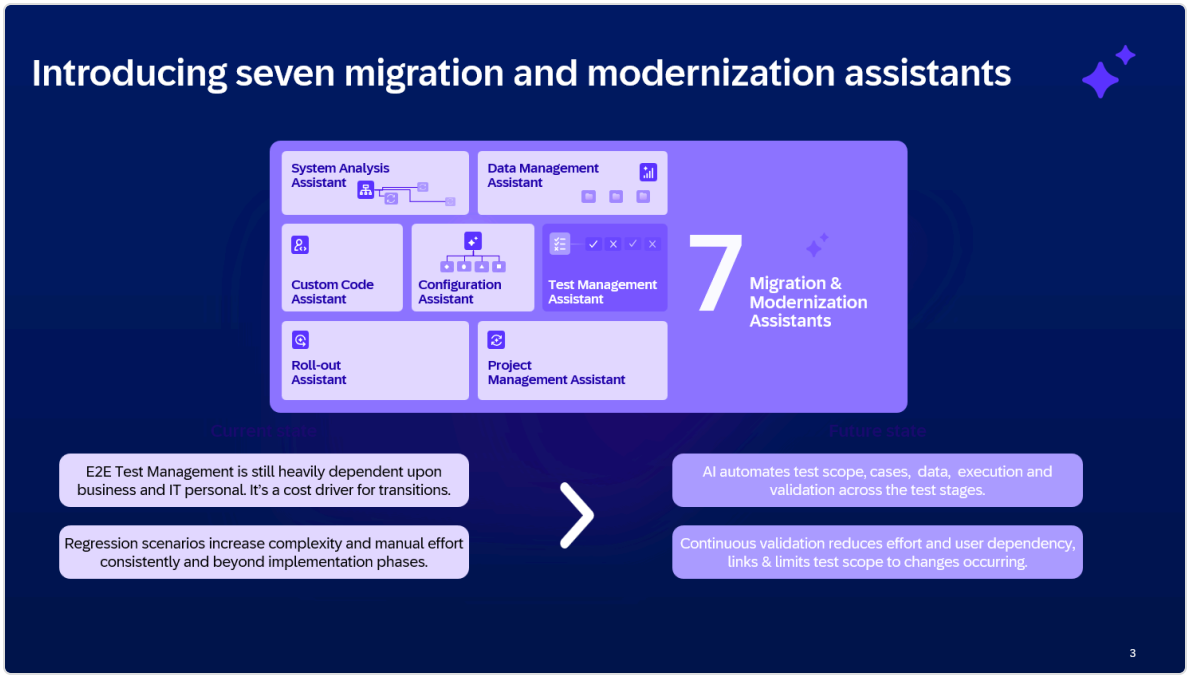
Agenda

#	Section	Topic
–	Introduction	SAP Migration & Modernization Assistants: context
–	Introduction	Test Management Assistant: agent architecture overview
0	Getting Started	Log In & Orient Yourself · Steps 1–4C
1	Generate Requirements	Turn an interview transcript into requirements with AI · Step 5
2	Scope with AI	Scoping agent; compare with manual scoping · Steps 6–10
3	Plan with AI	Planning agent; HITL Gate 2; review · Steps 11–14
4	Generate Test Cases	TC generation agent; quality review; parameters · Steps 15–18
5	Generate Test Data	Data agent; classify & explore; dependencies · Steps 19–23
6	Recap & Outlook	Time saved; roadmap; key takeaways · Steps 24–26

Introduction – SAP Migration & Modernization Assistants

> **Presenter narration:** Before we jump into the hands-on, a quick word on context, because what you're about to use is not an isolated feature.

SAP Cloud ALM Joule is the home of the **Test Management Assistant**: one of **seven** AI-powered assistants SAP is building for Migration and Modernization scenarios:



Why AI matters for test management specifically:

Current State	Future State
E2E Test Management is still heavily dependent upon business and IT personal; it's a cost driver for transitions.	AI automates test scope, cases, data, execution and validation across the test stages.
Regression scenarios increase complexity and manual effort consistently and beyond implementation phases.	Continuous validation reduces effort and user dependency, links & limits test scope to changes occurring.

> The Test Management Assistant is the one you're working with today. It sits at the quality gate of every migration, and as you'll see, it's where AI can compress days of manual work into minutes of guided review.

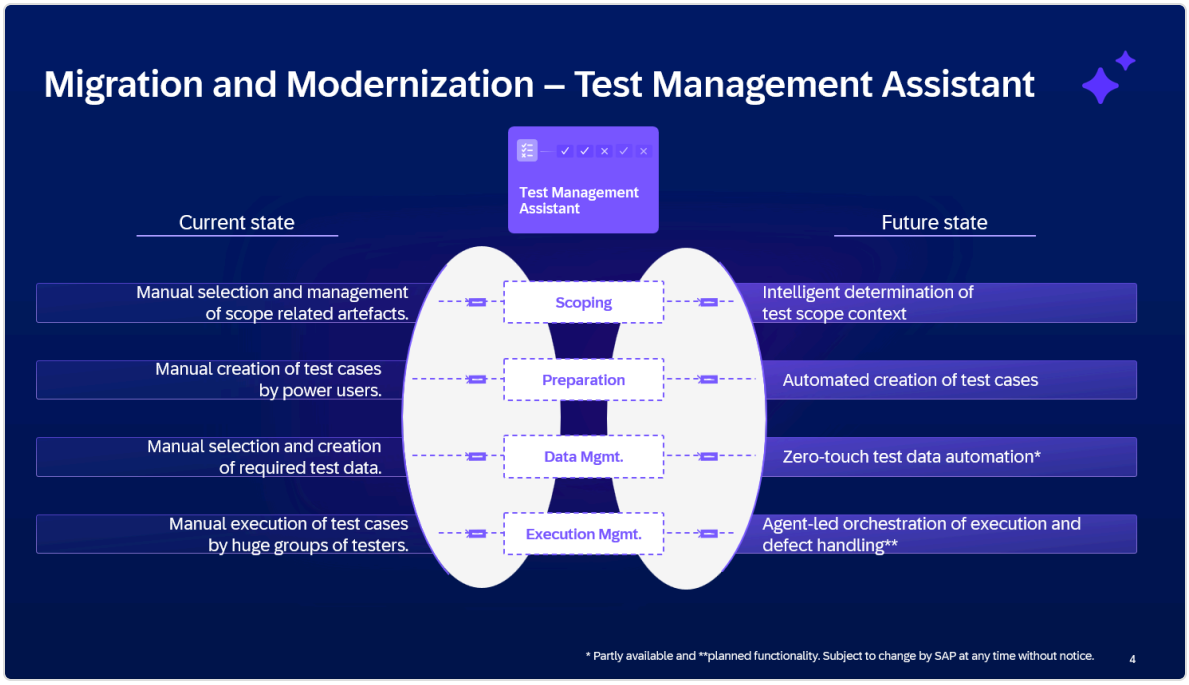
Introduction – Test Management Assistant: Agent Architecture

> **Presenter narration:** The Test Management Assistant is not a single AI feature; it's a pipeline of four specialized agents, each transforming manual work into an AI-driven capability.

The challenge each agent addresses:

Current State (Manual)	Agent	Future State (AI-driven)
Manual selection and management of scope-related artefacts	**Scoping**	Intelligent determination of test scope context
Manual creation of test cases by power users	**Preparation**	Automated creation of test cases
Manual selection and creation of required test data	**Data Mgmt.**	Zero-touch test data automation *(partly available)*
Manual execution of test cases by huge groups of testers	**Execution Mgmt.**	Agent-led orchestration of execution and defect handling *(planned)*

> **Today's hands-on covers the first three agents**: Scoping, Preparation, and Data Management. Execution Management is on the roadmap.
>
> ** Partly available and **planned functionality. Subject to change by SAP at any time without notice.*


Migration and Modernization – Test Management Assistant
Test Management Assistant
Current state
Future state
Manual selection and management of scope related artefacts.
Scoping
Intelligent determination of test scope context
Manual creation of test cases by power users.
Preparation
Automated creation of test cases
Manual selection and creation of required test data.
Data Mgmt.
Zero-touch test data automation*
Manual execution of test cases by huge groups of testers.
Execution Mgmt.
Agent-led orchestration of execution and defect handling**
* Partly available and **planned functionality. Subject to change by SAP at any time without notice.
4

Chapter 1 – Orientation

Session: TM261 – Agent-Led Test Management: Hands-on Lab with Joule in SAP Cloud ALM
Date: Thursday, 29 October 2026 · SAP TechEd 2026, Berlin
Presenters: Christina Pickhardt · Niclas Weidner · Tobias Keller
Scenario: Make-to-Stock Production – Discrete Manufacturing (BJ5)

Your Story Today

You are a **Test Business Process Expert** at a company that just activated SAP S/4HANA Scope Item BJ5: Make-to-Stock Production – Discrete Manufacturing. Your task: **prepare test coverage for the new process** from scratch.

In this hands-on you'll work with AI agents in SAP Cloud ALM Test Management to speed up every stage, from requirements to test data, dramatically reducing the manual effort that would normally take days.

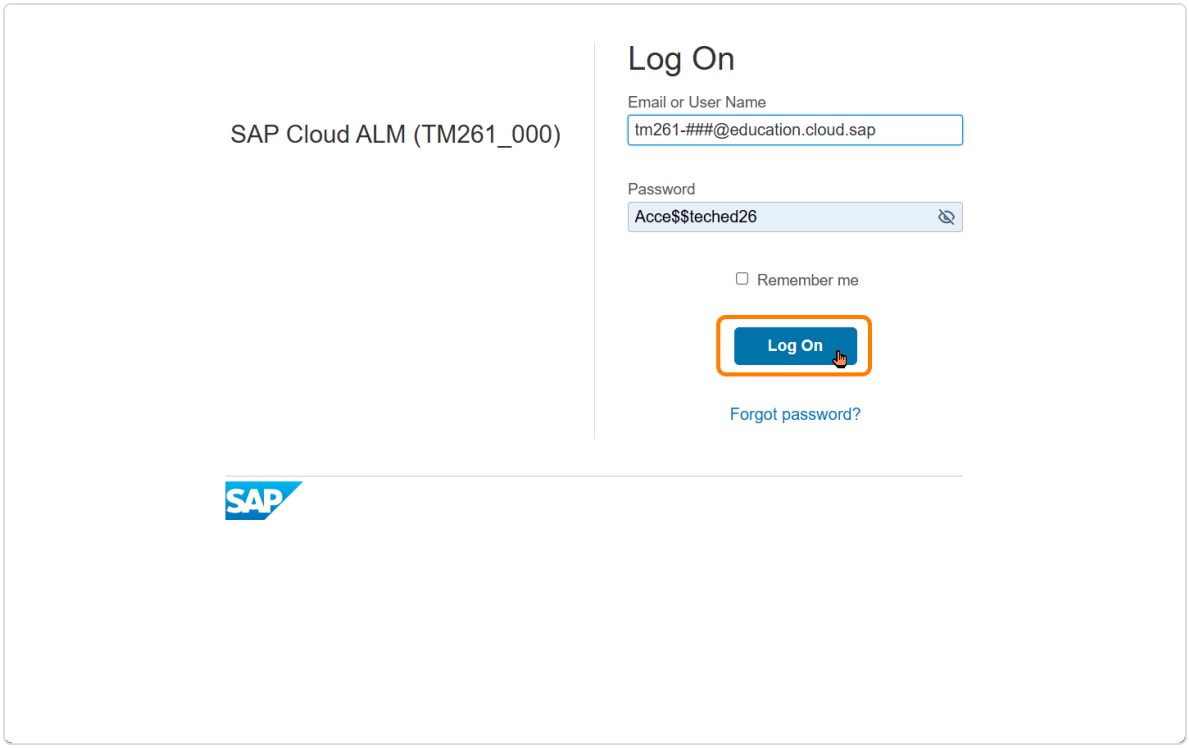
1.1 – Log In to SAP Cloud ALM

You are a business and process expert at a large multinational corporation. It's the start of your work day, and you have an important task on your agenda: preparing test coverage for a newly activated SAP process.

Open your browser. The SAP Cloud ALM login page is already loaded.

> **Your credentials:**
>
> | Field | Value |
> |---|---|
> | User | `tm261-###@education.cloud.sap` |
> | Password | `Acce$$teched26` |
>
> Replace **###** with your **workstation number** (shown on your desk), for example: 001, 018, 042. This number is yours for the entire session; it identifies your personal data and work in the shared system.

1. Confirm the tenant name on the left reads **SAP Cloud ALM (TM261_000)**: this is the shared demo system for this session.
2. Enter your username and password, then click **Log On**.



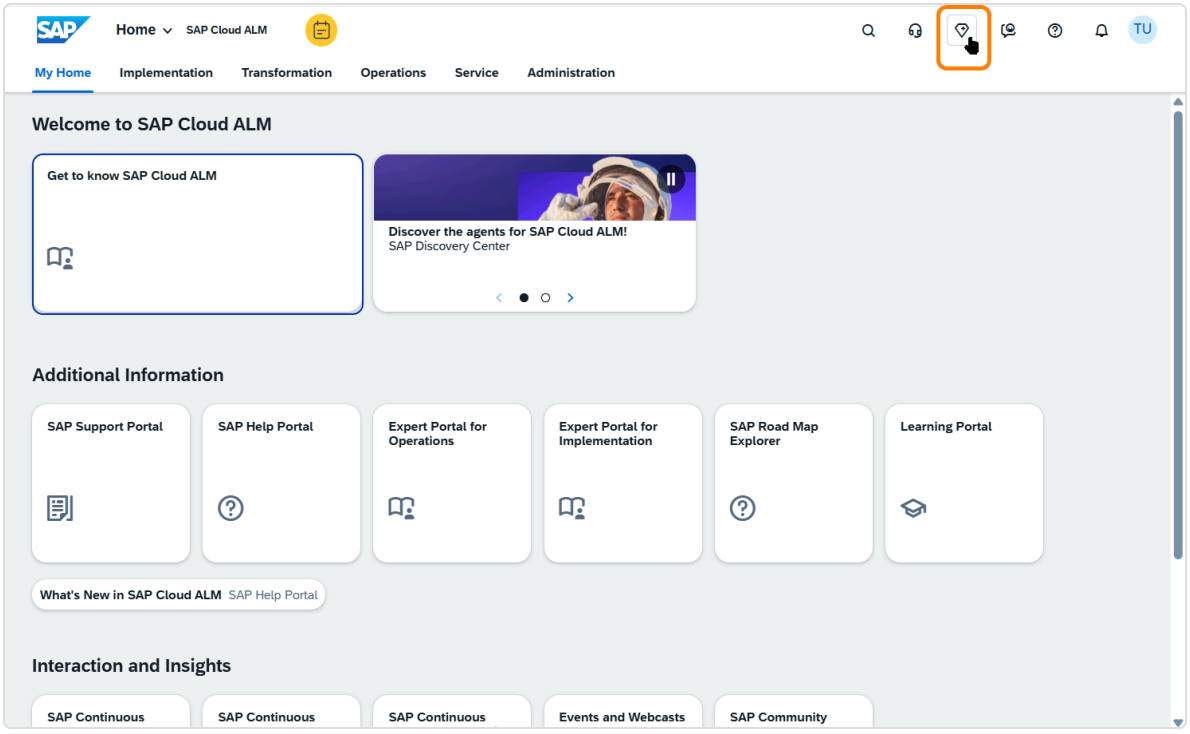
01 – Login to SAP Cloud ALM

1.2 – Open Joule

After logging in you land on the SAP Cloud ALM home page. Time to call your most valuable tool: **Joule**.

Joule is a chat-style AI interface built into SAP Cloud ALM. You'll use it throughout this session to interact with AI agents, ask questions, and drive the test preparation workflow; all in natural language.

1. Look for the **diamond icon** (◇) in the upper-right area of the screen and click it to open Joule.

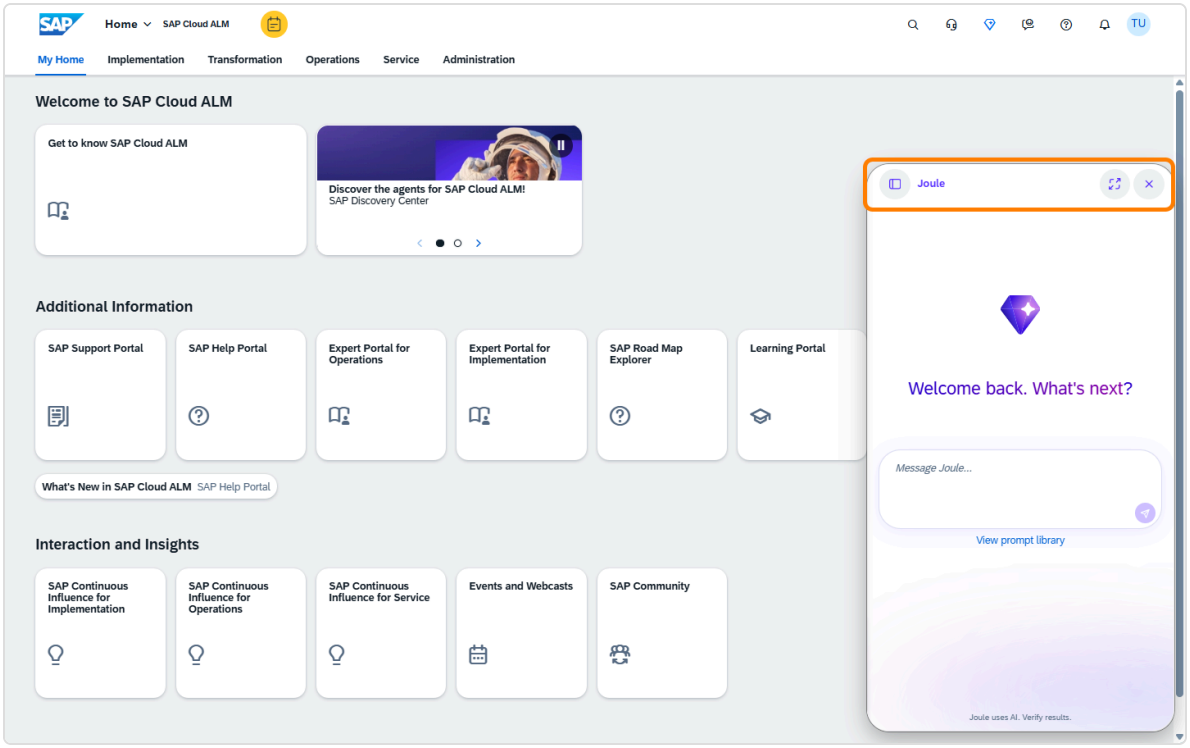


02 – SAP Cloud ALM home page

1. Joule opens as a panel that slides over the application screen; it's always available, regardless of where you are in SAP Cloud ALM.

The panel header gives you three controls:
- **Sidebar icon**: manage multiple concurrent conversations (you can run several Joule chats in parallel, e.g. one per topic or agent)
- **Expand icon**: open Joule in full-screen mode
- **Close (×)**: close the panel

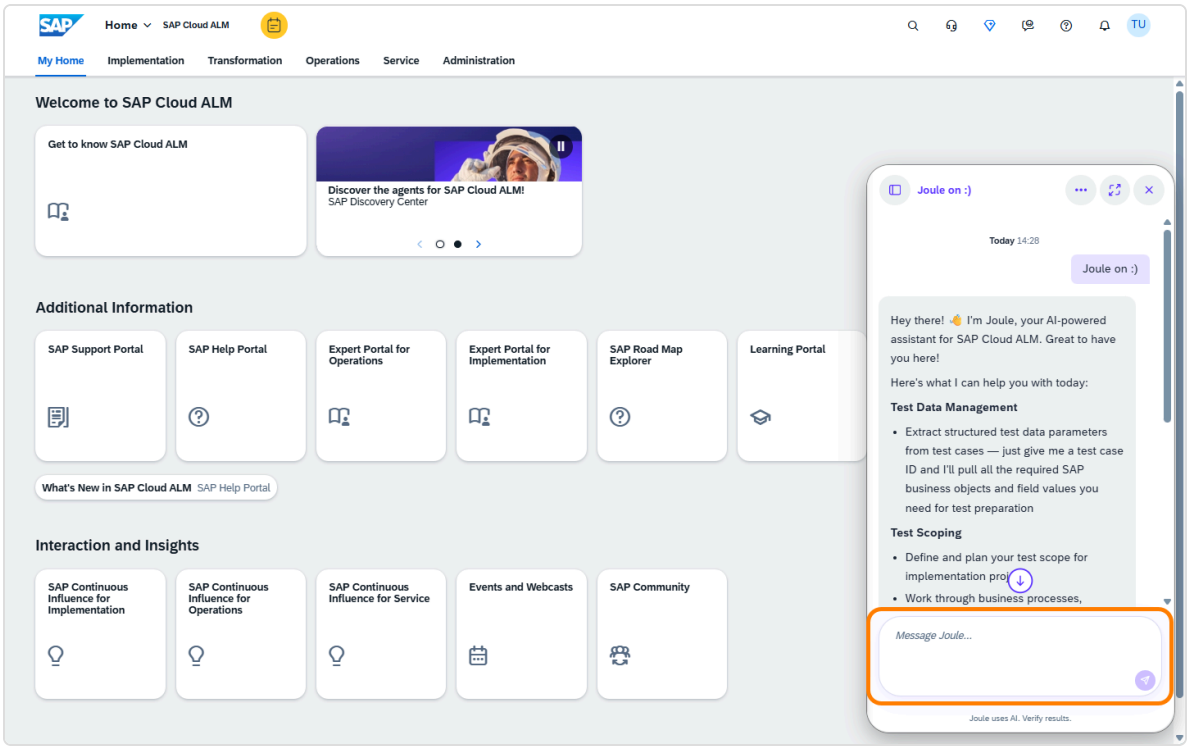
> **Good to know:** Joule is stateful. Close the panel, navigate elsewhere, reopen it; your conversation is still there exactly as you left it.



03 – Joule panel open

1. Wake up Joule with a simple message in the input box at the bottom; just type naturally, for example: **"Joule on :)"** or any greeting.

Joule responds with an introduction of its capabilities. Scroll through its response; you'll see areas like **Test Data Management**, **Test Scoping**, **Test Planning**, and **ALM Project Setup**. These are exactly the capabilities you'll use today.



04 – Joule greeting and capabilities

1.3 – Jump Ahead: What Happens Without Preparation? *(Optional)*

> **Optional step**: try this if you want to see what happens when you skip straight to test case generation without proper setup.

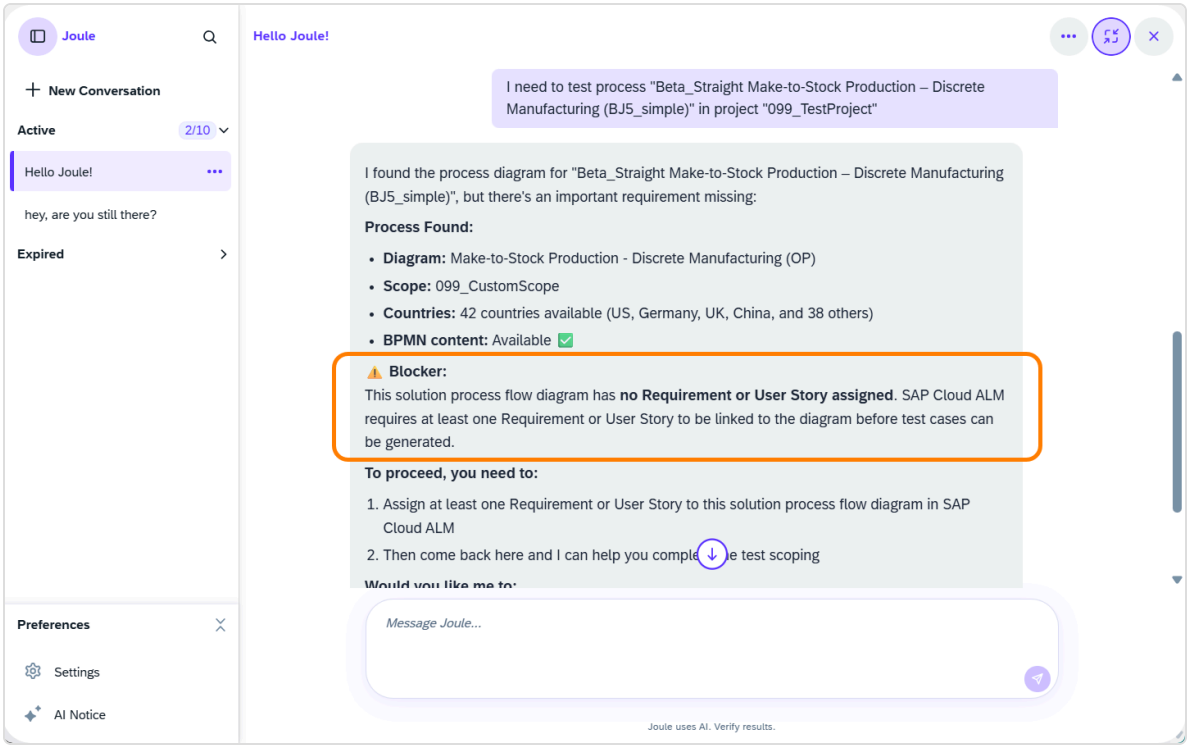
Let's see what Joule does when asked to work with incomplete input.

1. In Joule, prompt it to generate test cases directly; specifying the process and project:

> *"I need to test process "Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)" in project "###_TestProject""*
>
> Replace **###** with your workstation number.

Joule navigates to the process in your project and analyses its content, but it hits a problem.

> **Tip:** Click the **expand icon** (⤢) in the Joule panel header to open Joule in full-screen mode: much easier to read Joule's responses, especially for longer agent outputs. The left sidebar shows your conversation list: up to **10 concurrent conversations** can be active at the same time, with older ones moved to **Expired**.



05 – Joule blocked: no requirement or user story

> ### ⚠️ Blocker: No Requirement or User Story Assigned
>
> Joule found the process and confirmed the BPMN diagram is available, but it immediately flags a blocker: **no Requirement or User Story is linked to the process**. SAP Cloud ALM requires at least one to be assigned before test cases can be generated.
>
> This is intentional. **Garbage in, garbage out**: even for AI agents. The quality of the output depends directly on the quality of the input. Before Joule can do useful work, we need to enrich the process with requirements and user stories.
>
> That's exactly what we'll do in the next steps.

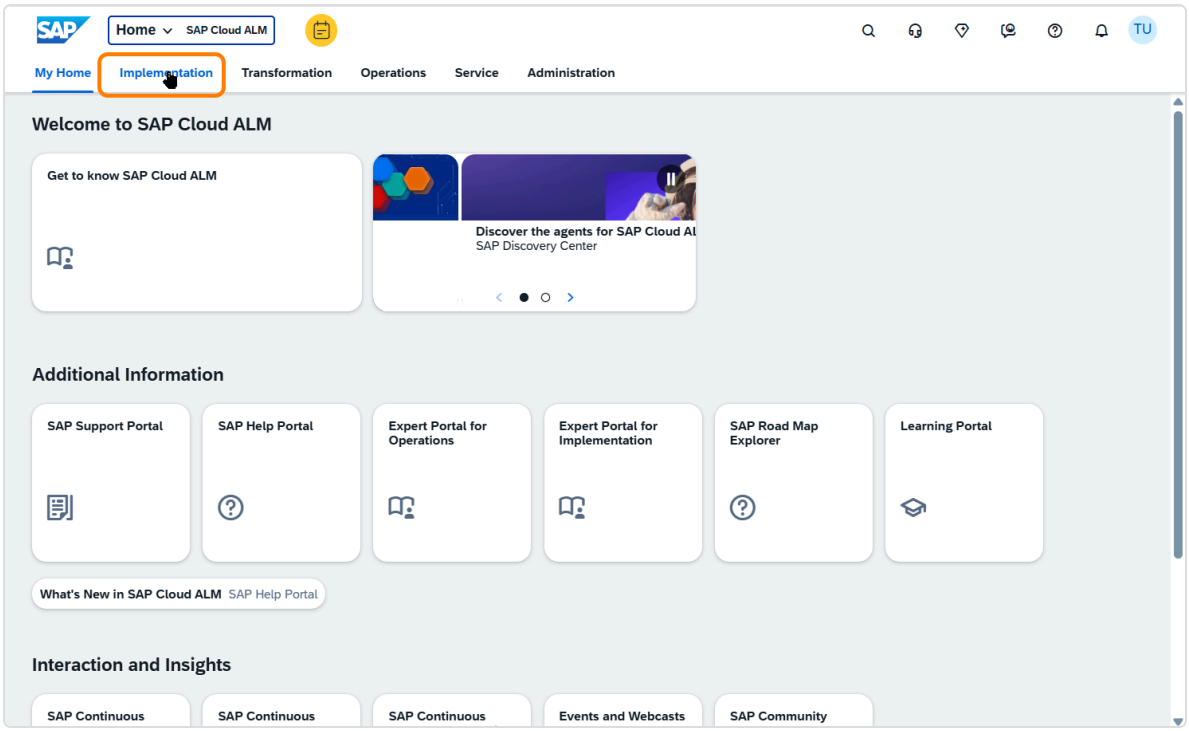
Chapter 2 – Build Your Input

Before the AI agents can do useful work, the process needs to be populated with the documentation they'll work from: a BPMN diagram, user stories, and a requirements document. In this chapter you'll navigate to your project, assign pre-prepared user stories, and generate a requirements document from an expert interview transcript.

2.1 – Navigate to Your Project

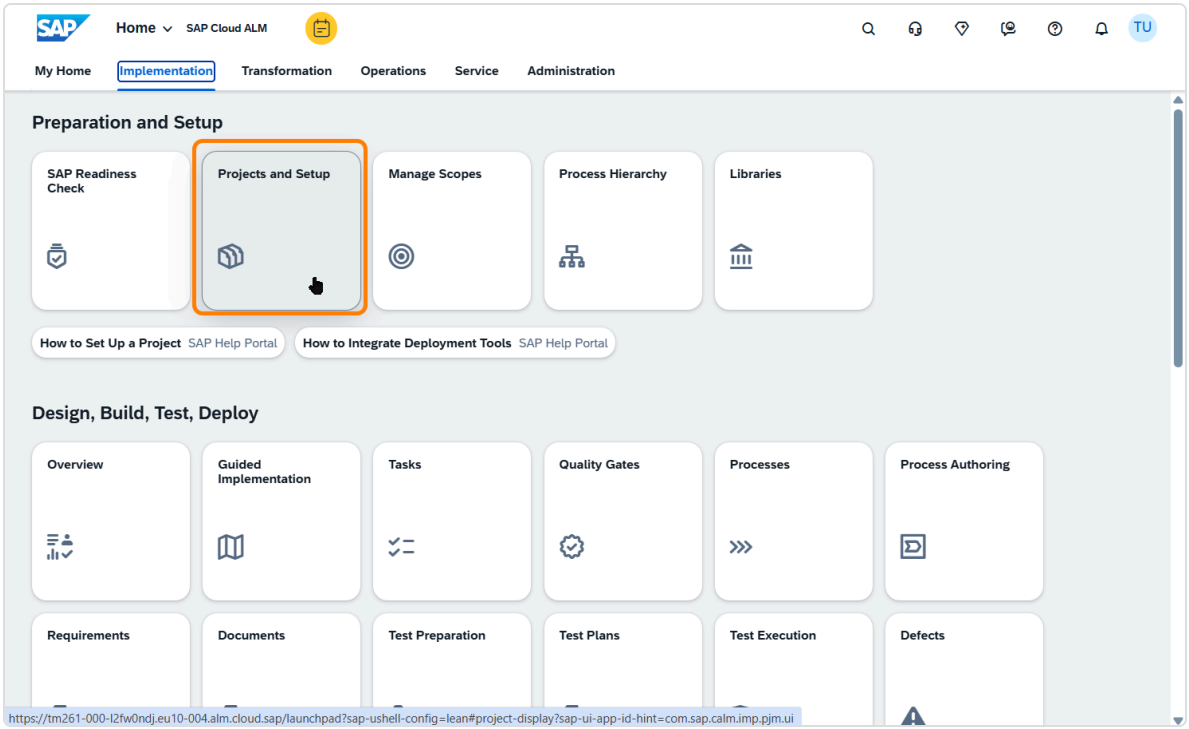
Time to find our process. Close the Joule panel first (click × in the panel header), then navigate to the project.

1. Back on the home screen, click the **Implementation** tab in the top navigation bar. This is the launchpad for all test management related apps, and much more: requirements, process authoring, test preparation, test plans, test execution, defects.



06 – Implementation tab

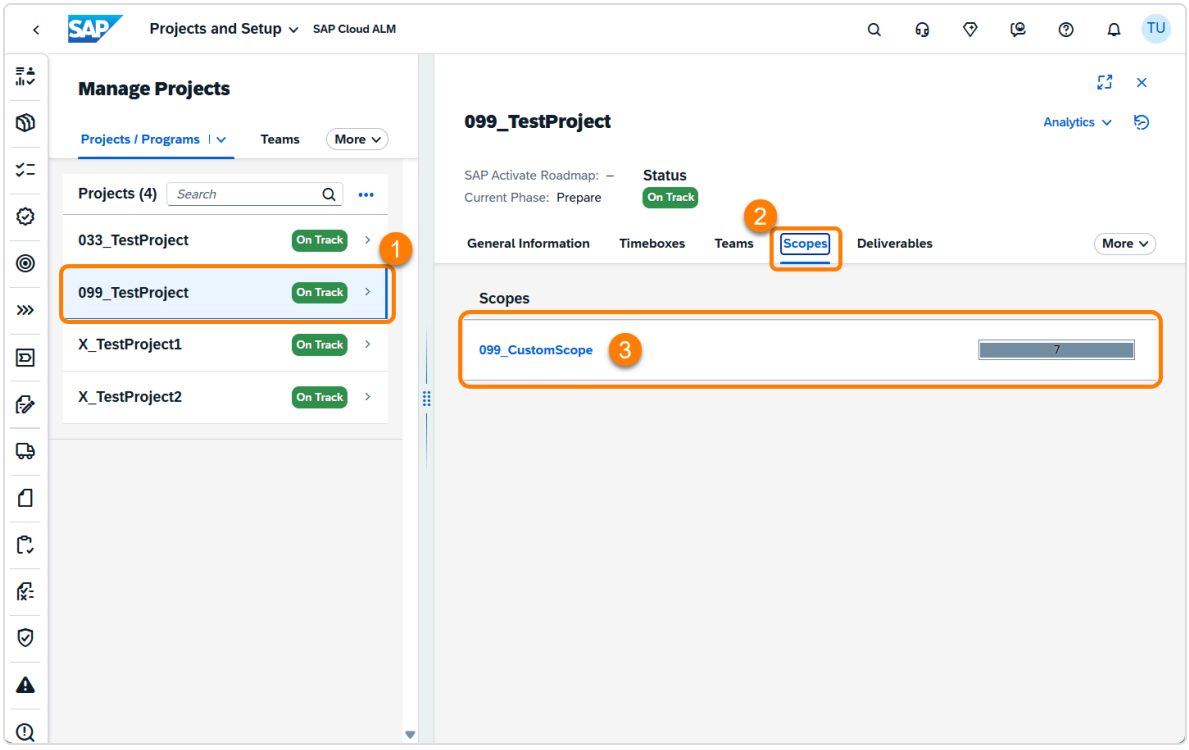
1. Under **Preparation and Setup**, click the **Projects and Setup** tile.



07 – Projects and Setup tile

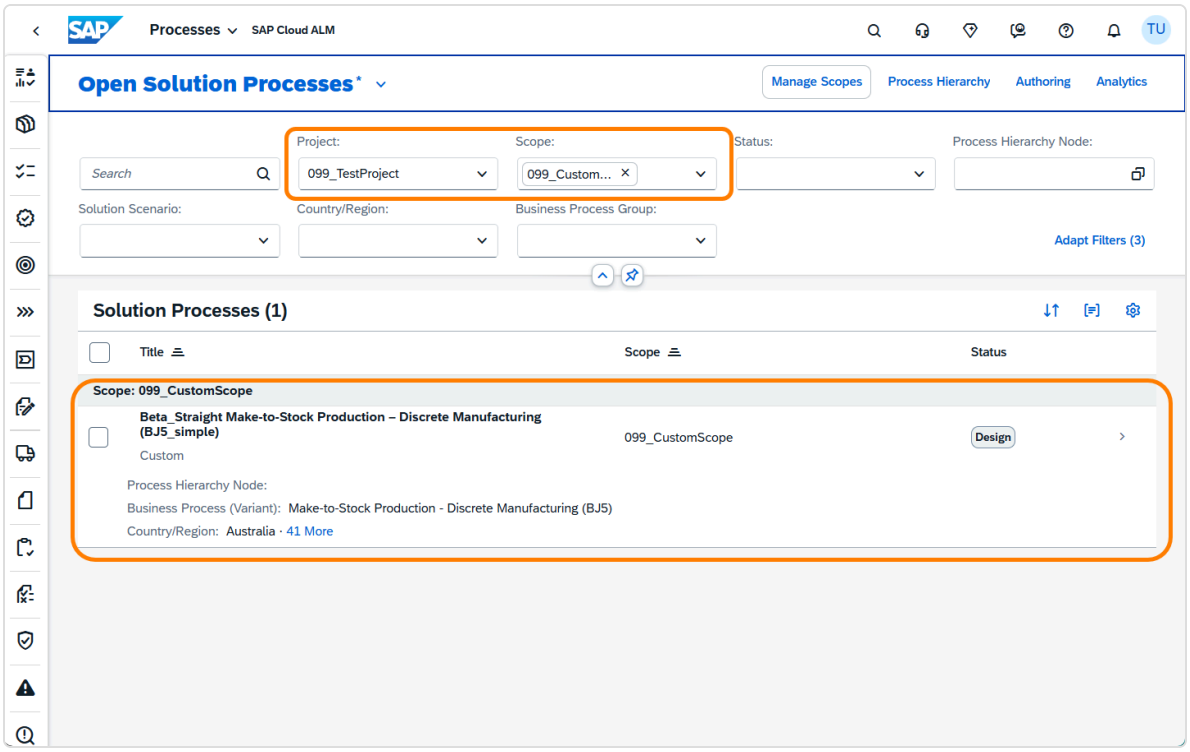
1. In the **Manage Projects** list, locate your project: `###_TestProject` (replace ### with your workstation number). Click it to open the project details.

2. In the project detail view, click the **Scopes** tab. You'll see one scope: `###_CustomScope`: this is the container for all the process content you'll work with today. Click on it.



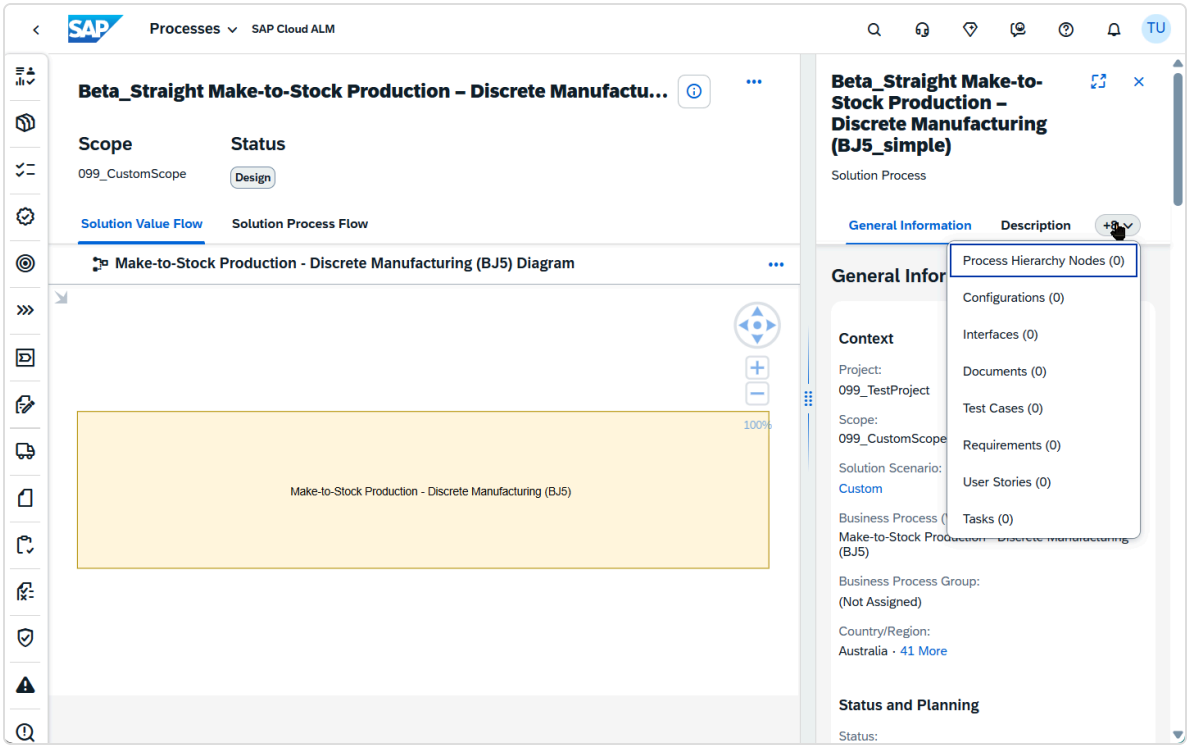
08 – Project list and scope

1. The view switches to **Open Solution Processes**, with the filter pre-set to your project and scope. You should see exactly one process: `Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)` : your process for today. Click on it to open its details.



09 – BJ5 process in the scope

1. The process detail view opens. On the right-hand panel, next to the **General Information** and **Description** tabs, click **+8** to expand the full list of content types that can be linked to this process. Everything is at zero: no requirements, no user stories, no test cases. The process diagram on the left is also empty; no flows, no steps. This is your starting point.



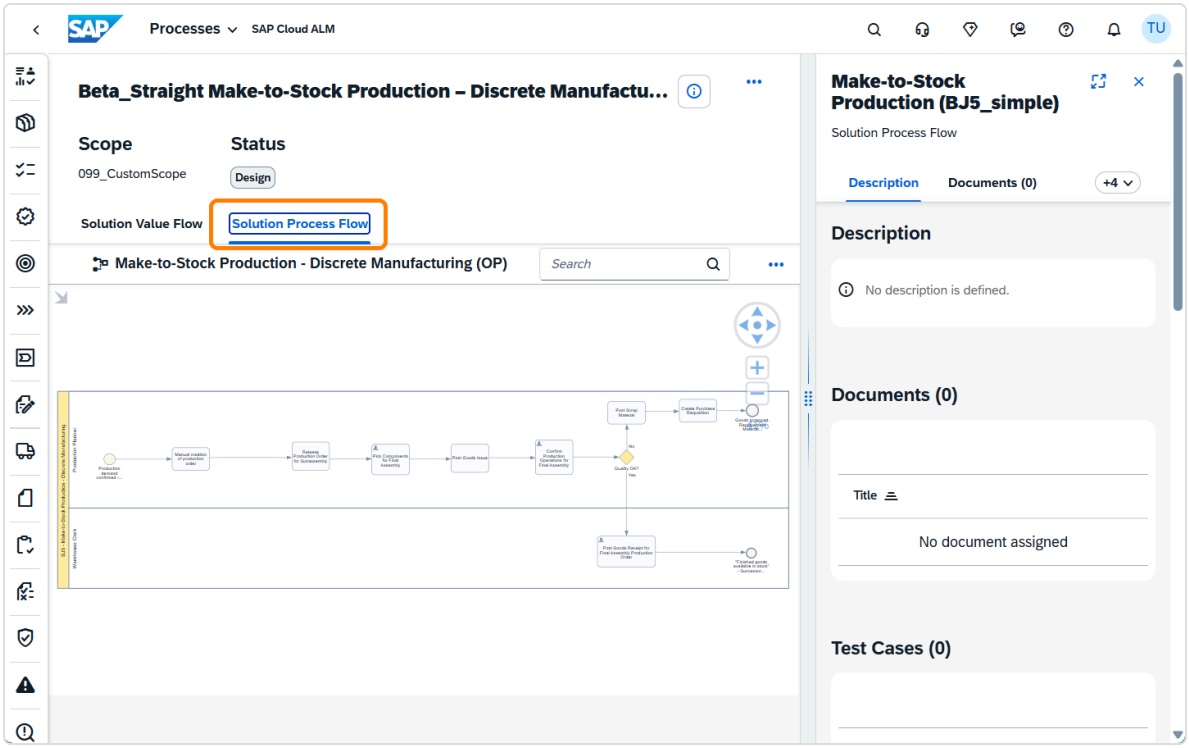
10 – Process detail: all content at zero

> **This is the root cause** of Joule's rejection in Step 1.3. With no requirements, no user stories, and no process content, there is nothing for the AI agents to work from. Before we can generate anything meaningful, we need to populate this process, and that's exactly what the next steps will do.

1. Now click the **Solution Process Flow** tab (next to Solution Value Flow). Here you'll find something more promising: a BPMN process diagram showing the BJ5 flow across two swimlanes; Production Planner and Warehouse Clerk.

This diagram was imported into SAP Cloud ALM from a process modeling tool such as **SAP Signavio**. It is natively stored here; a direct live integration with Signavio is planned as a future capability.

> **But here's the catch:** the diagram alone is not sufficient for the AI agents to generate high-quality scope, test plan, and test cases. It provides the process skeleton, but without requirements, user stories, and linked documentation, the agents can't derive meaningful test coverage from it.
>
> Feel free to explore the diagram. You can even make small changes, but don't alter it too dramatically, or the AI agents may flag inconsistencies between the diagram and the other content you'll add next.



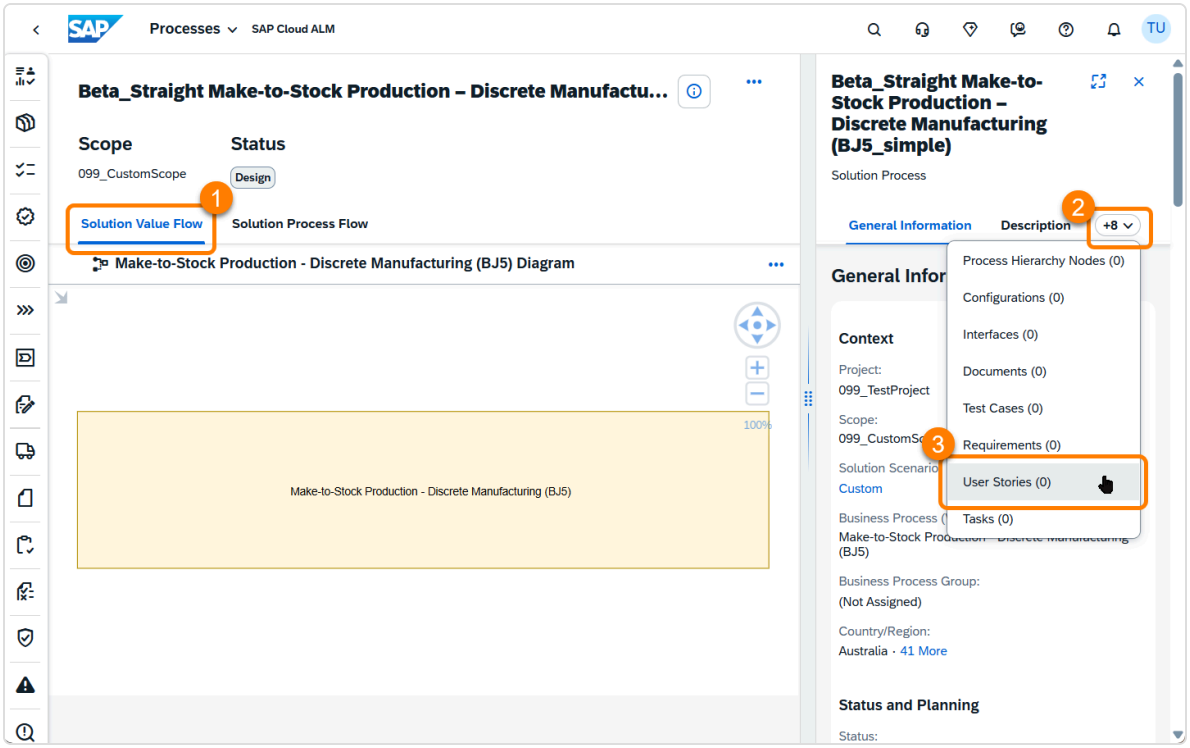
11 – BJ5 process flow diagram

> **In short:** A process structure exists and the diagram provides the skeleton. However, the process still lacks enriched content the AI agents need to do useful work. That's exactly what the next steps will fix.

2.2 – Assign User Stories

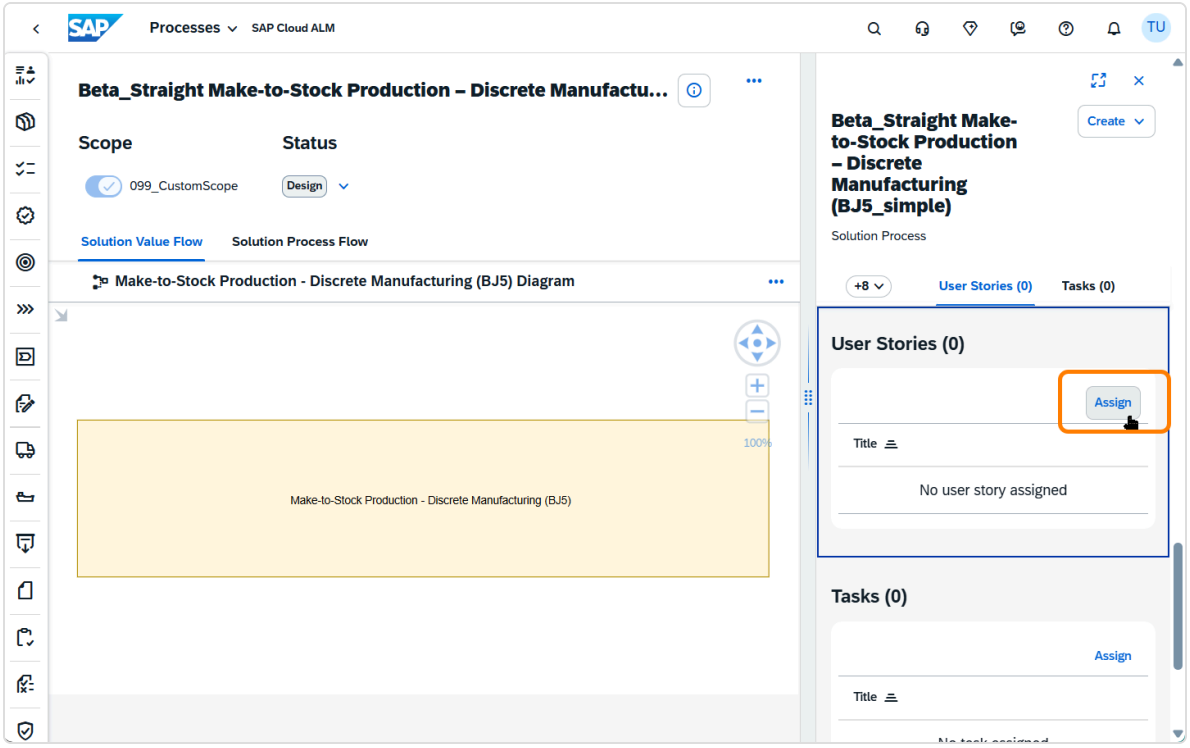
To give the AI agents enough to work with, we need to link supporting content to the process. Let's start with the user stories.

1. Make sure you are on the **Solution Value Flow** tab. On the right-hand panel, click **+8** to open the dropdown and select **User Stories**: this brings the User Stories section into view.



12 – Opening User Stories section

1. The **User Stories** section is now visible in the right panel: currently empty ("No user story assigned"). Click the **Assign** button to open the selection dialog. It comes pre-populated with user stories prepared for the BJ5 process.

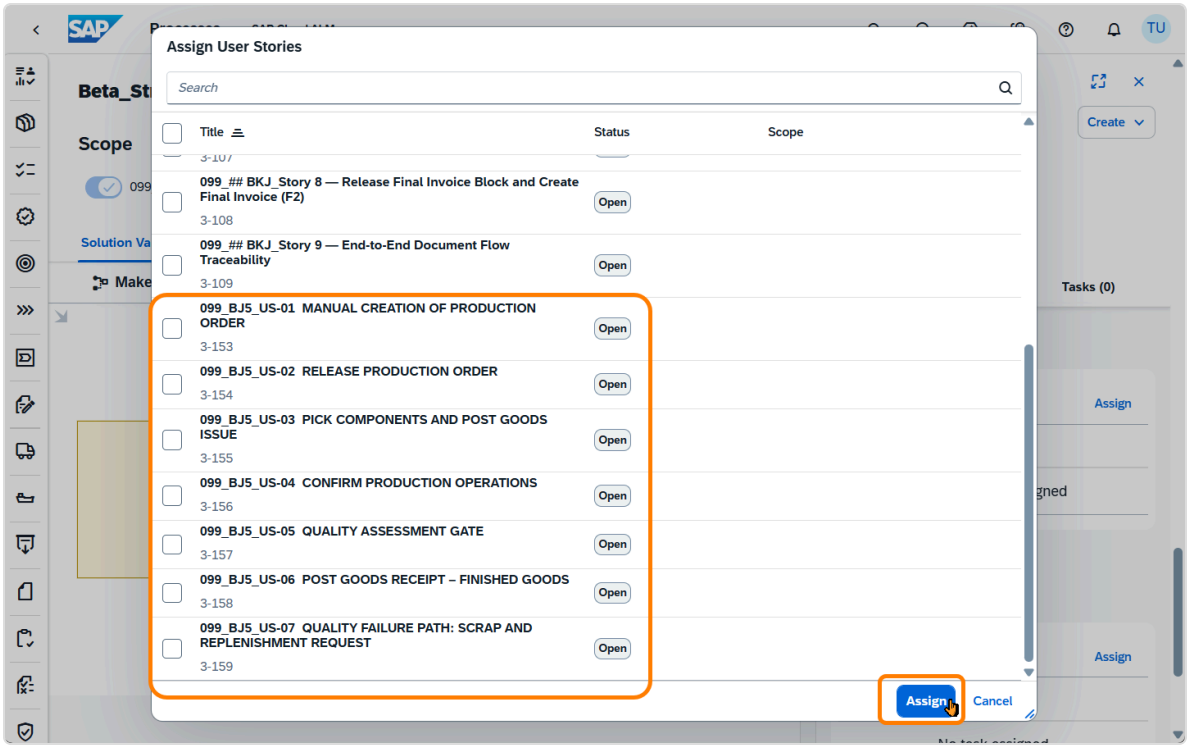


13 – Assign User Stories dialog

1. The **Assign User Stories** dialog opens. Scroll down to find the 7 BJ5 user stories (prefixed **BJ5_US-**) and select all of them:

#	Story
1	Manual Creation of Production Order
2	Release Production Order
3	Pick Components and Post Goods Issue
4	Confirm Production Operations
5	Quality Assessment Gate
6	Post Goods Receipt – Finished Goods
7	Quality Failure Path: Scrap and Replenishment Request

Once all 7 BJ5 stories are selected, click **Assign**.



14 – Selecting all 7 BJ5 user stories

1. After returning to the process, feel free to open one of the user stories to inspect it. Each story is fully defined with **story points**, the **user story** narrative ("As an [role], I want to... so that..."), and granular **acceptance criteria**: exactly the kind of structured input the AI agents need.

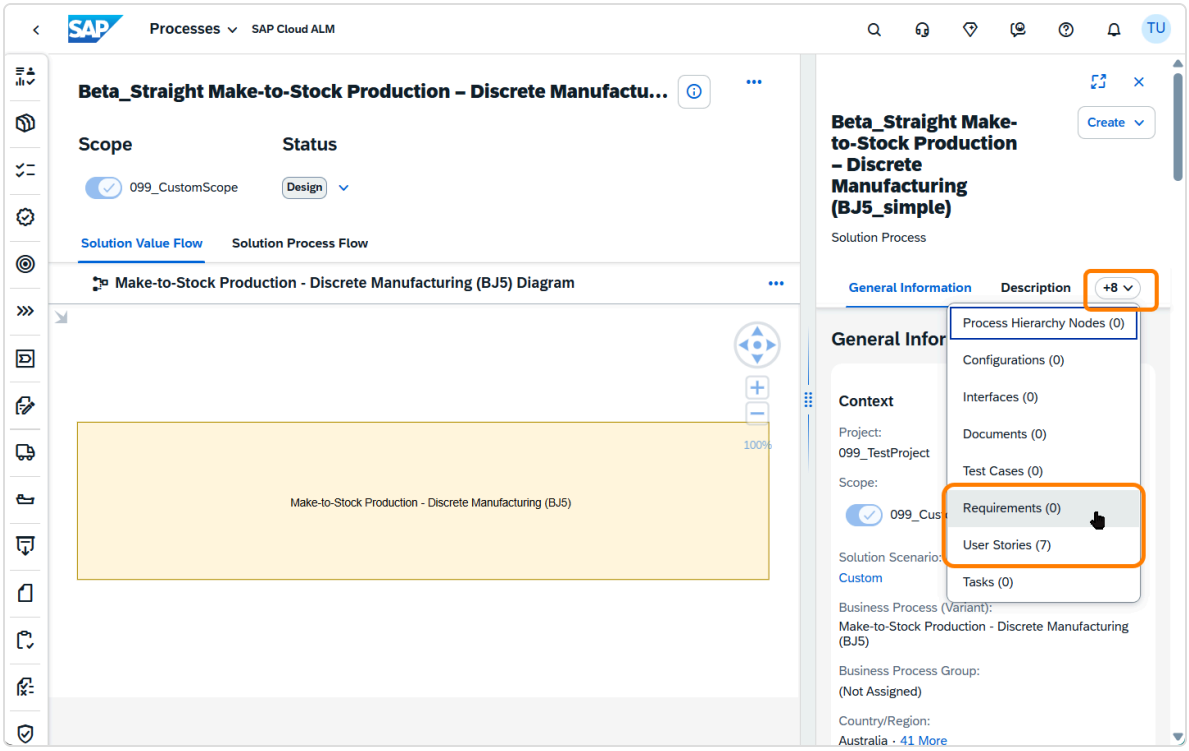
▶ 📷 Example: User Story 1: Manual Creation of Production Order

When done, click the **back arrow** (←) at the top left of the screen to return to the process view.

1. Back in the process, open the **+8** dropdown again; **User Stories** now shows **(7)**. The assignment worked.

But look at **Requirements (0)**. That entry is still empty, and it's arguably the most important input for the AI agents. A well-formed requirements document gives the agents the business rules, constraints, and acceptance detail they need to generate meaningful test coverage.

Here's the good news: you were recently in a structured interview with a BJ5 process expert, walking through the process step by step in detail. Your colleague promised to post the transcript in SAP Cloud ALM for further use. Time to retrieve it and turn it into a requirements document.



16 – Requirements count still zero

Summary

Chapters 1 and 2 (so far):
- [x] Logged in and opened Joule
- [x] BJ5 process located in `###_TestProject / ###_CustomScope`
- [x] 7 BJ5 user stories assigned to the process
- [ ] Requirements document: still missing; covered in step 2.3

Continue to Chapter 2 – Build Your Input (continued)

2.3 – Generate Requirements from an Interview Transcript

Prerequisite: You have completed Chapter 2 – steps 2.1 and 2.2; the BJ5 process is set up with user stories.

What You'll Do

In this step you'll use the SAP Cloud ALM Requirements app to generate a requirements document from an interview transcript. This is your first AI-assisted step; instead of drafting requirements manually, the AI reads an expert interview and structures it for you.

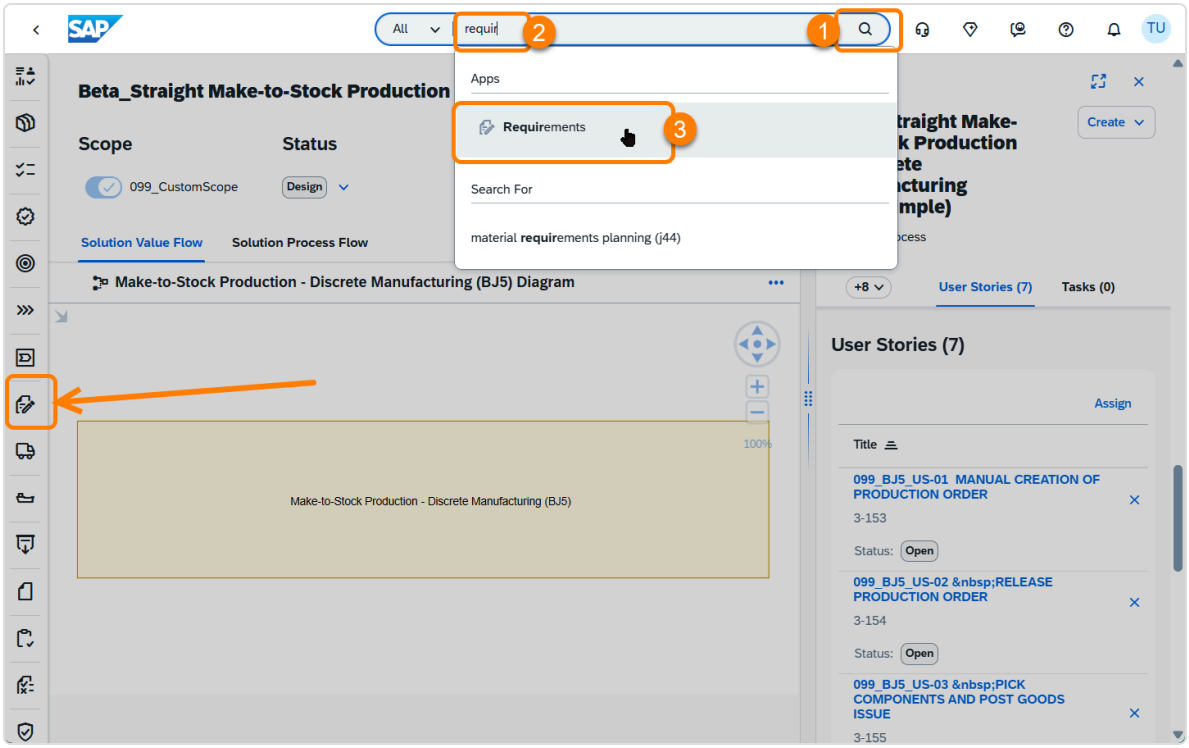
> **Why this matters:** Good requirements are the foundation for good test cases. Without them, the scoping and planning agents in later chapters won't have enough context to produce high-quality output. You'll see this contrast directly.

A requirements interview with a BJ5 process expert was recorded and transcribed for you. The transcript captures all the key business rules, process steps, and constraints discussed in the expert session.

2.3.1 – Open the Requirements App

Navigate to the **Requirements** app. There are two ways:

- **Search bar** (top center): type `require` and select **Requirements** from the results
- **Left toolbar**: click the quick-link icon that looks like a document being written on with a pen



01 – Requirements app

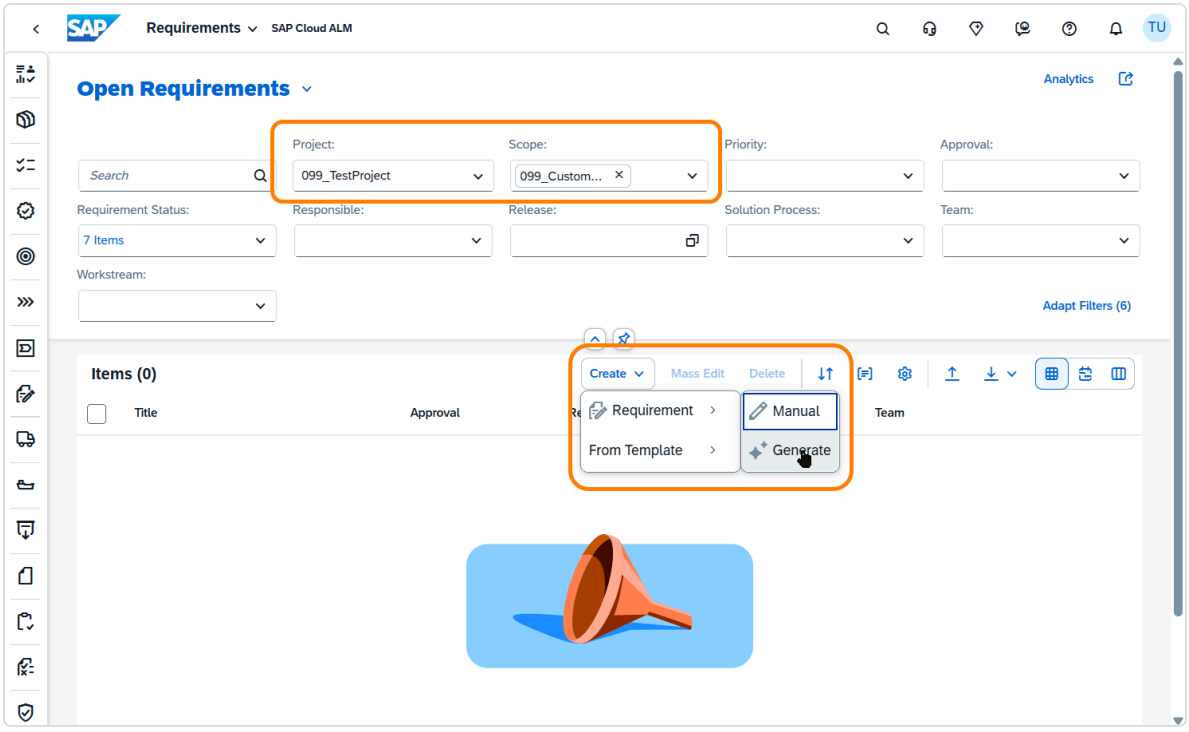
2.3.2 – Create a Requirement via Generate

1. The Requirements app opens with the filter pre-set to your project and scope; confirm both show your **workstation number** (`###_TestProject` / `###_CustomScope`). The list is empty: no requirements exist yet.

2. Click **Create**, then hover over **Requirement**: a sub-menu appears with two options:

3. **Manual**: write the requirement yourself
4. **Generate** ✨; let AI generate it from a source document

Click **Generate**.



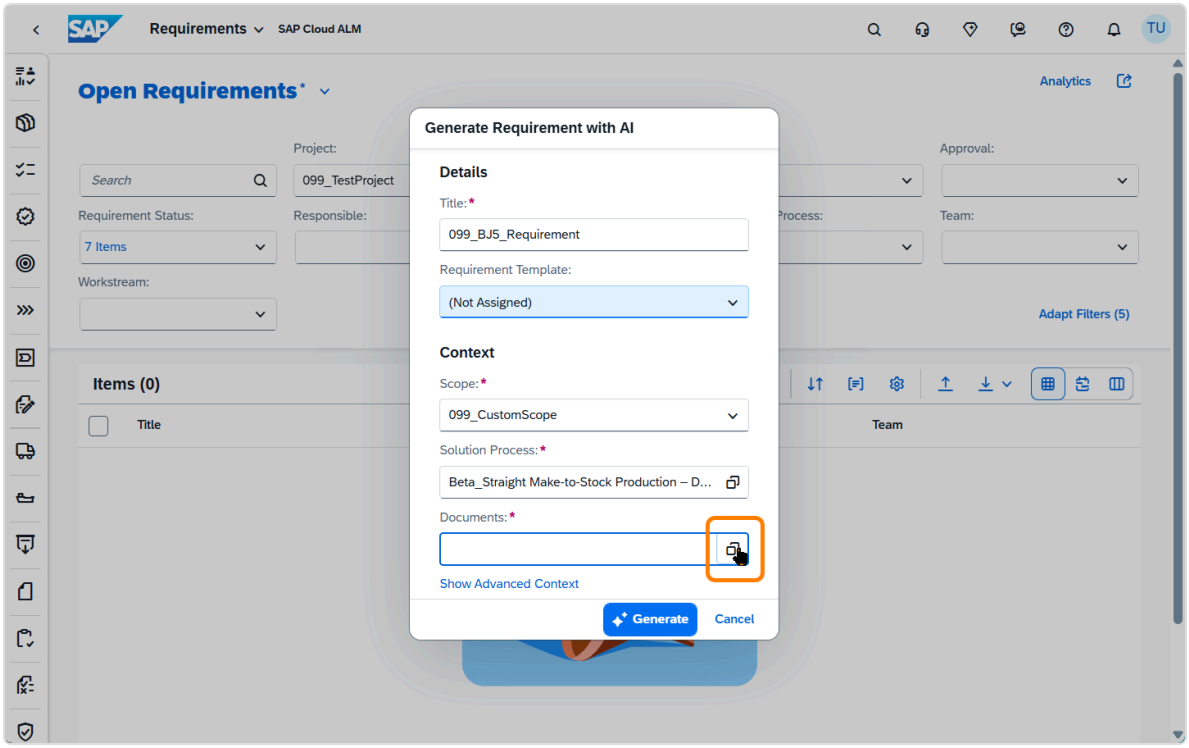
02 – Create → Generate menu

2.3.3 – Configure the Generation Dialog

The **Generate Requirement with AI** dialog opens. Fill in the fields:

Field	Value
Title	`###_BJ5_Requirement` (replace ### with your workstation number)
Requirement Template	(Not Assigned)
Scope	`###_CustomScope`
Solution Process	Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)
Documents	← click the selection icon to open the document picker

Click the **selection icon** next to the **Documents** field to choose the source document; the interview transcript your colleague posted.



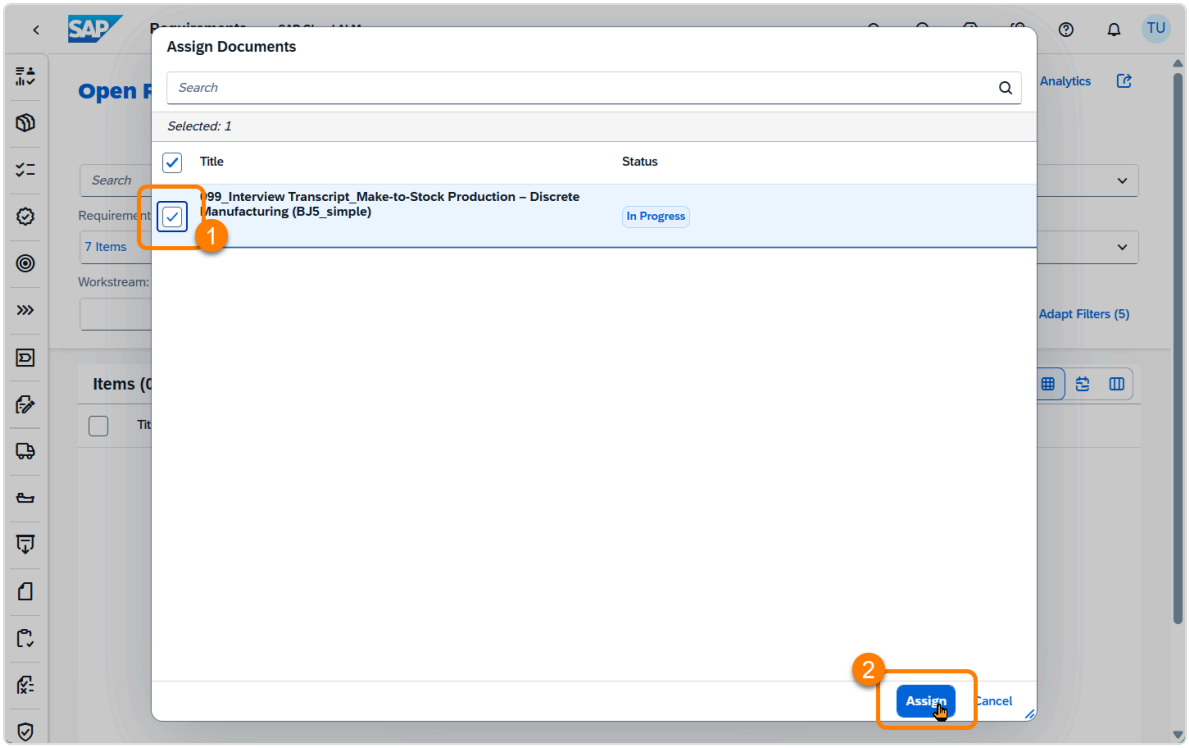
03 – Generate Requirement with AI dialog

2.3.4 – Select the Interview Transcript

The **Assign Documents** dialog opens, showing the documents available in your project. You should see the interview transcript your colleague posted:

`###_Interview Transcript_Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)`

Select it and click **Assign**.

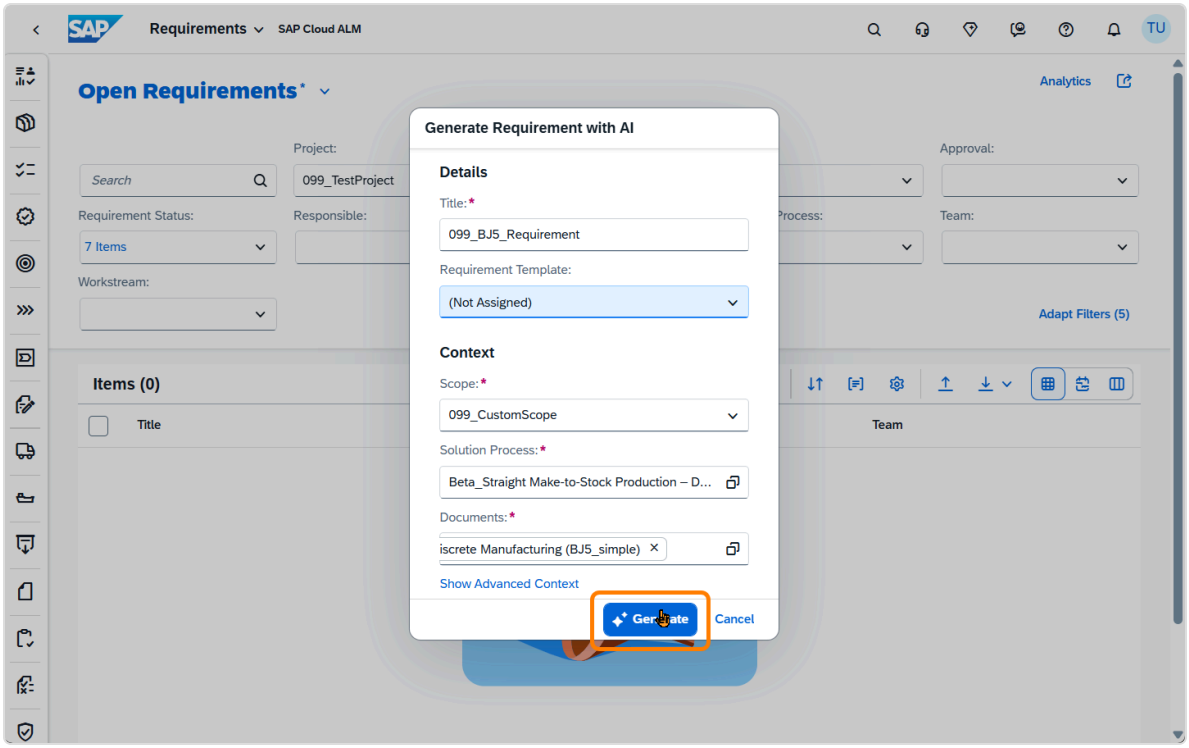


04 – Selecting the interview transcript

2.3.5 – Generate

Back in the **Generate Requirement with AI** dialog, the transcript now appears in the **Documents** field. Everything is set. Click **Generate**.

SAP Cloud ALM's built-in AI agent reads the full 30-minute conversation transcript and converts it into a structured requirements document; automatically. You'll be able to review the result in the next screen.



05 – Triggering AI generation

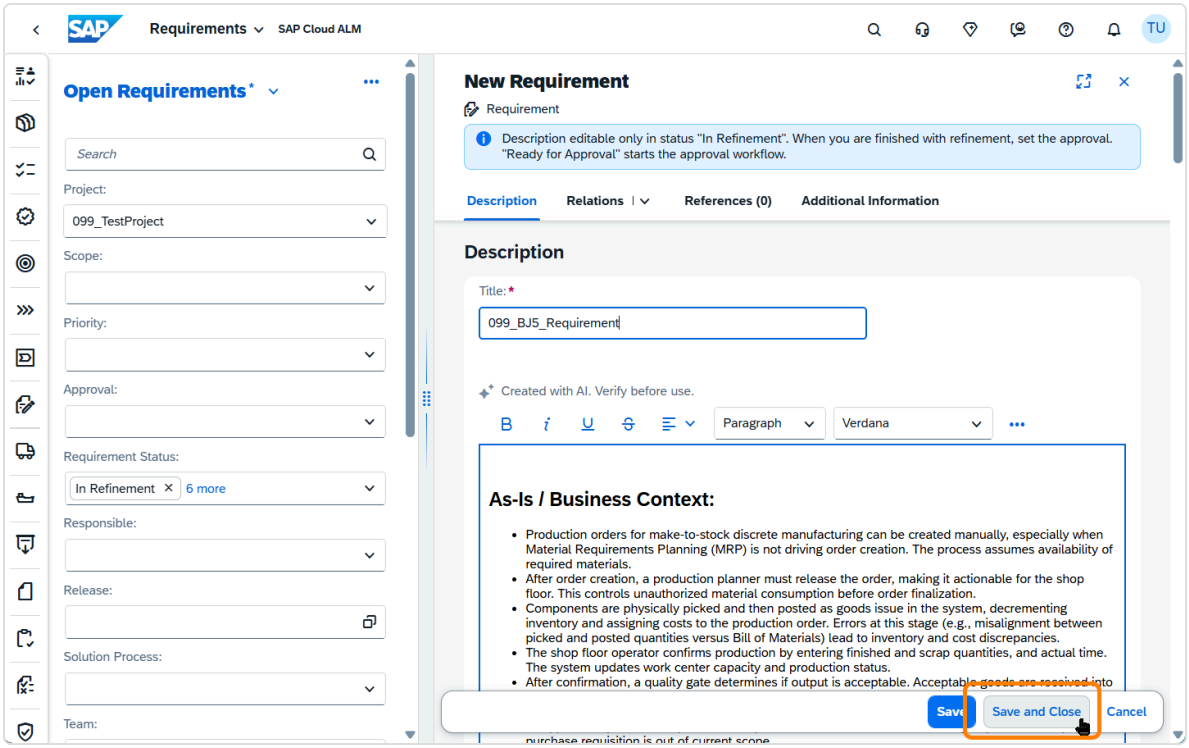
2.3.6 – Review and Save the Generated Requirement

The AI has created the new requirement. The detail view opens automatically.

A few things worth noting here:

- **Scroll up** to read the generated description; this is the structured requirements content extracted from your interview transcript.
- **Scroll down** to the **Solution Processes** section: the BJ5 process has been **automatically detected and linked**: no manual assignment needed. The AI recognised the context from the document.
- The info banner at the top reminds you that the description is editable while the status is *In Refinement*. Feel free to make adjustments if needed.
- At the bottom you can optionally fill in approver, priority, and other fields; skip this for now.

When satisfied, click **Save and Close**.



06 – Generated requirement: detail view

Summary

Chapter 2 complete. The process now has everything the AI agents need:
- [x] BPMN process diagram (from Signavio)
- [x] 7 user stories (assigned in Step 2.2)
- [x] Requirements document (generated in Step 2.3)

A 20-minute expert interview was converted into a structured requirements document, automatically linked to the BJ5 process. Time to head back to Joule and try what failed in Step 1.3; this time with proper input in place.

Continue to Chapter 3 – Scoping Agent

Chapter 3 – Scoping Agent

Prerequisite: You have completed Chapter 2; the BJ5 process has linked requirements and user stories.

What You'll Do

In this chapter you'll unleash the **Scoping Agent**: an AI agent that reads all your linked process documentation and proposes a structured set of test cases. While the agent works, you'll experience what manual scoping looks like, so you can appreciate the comparison.

3.1 – Trigger the Scoping Agent

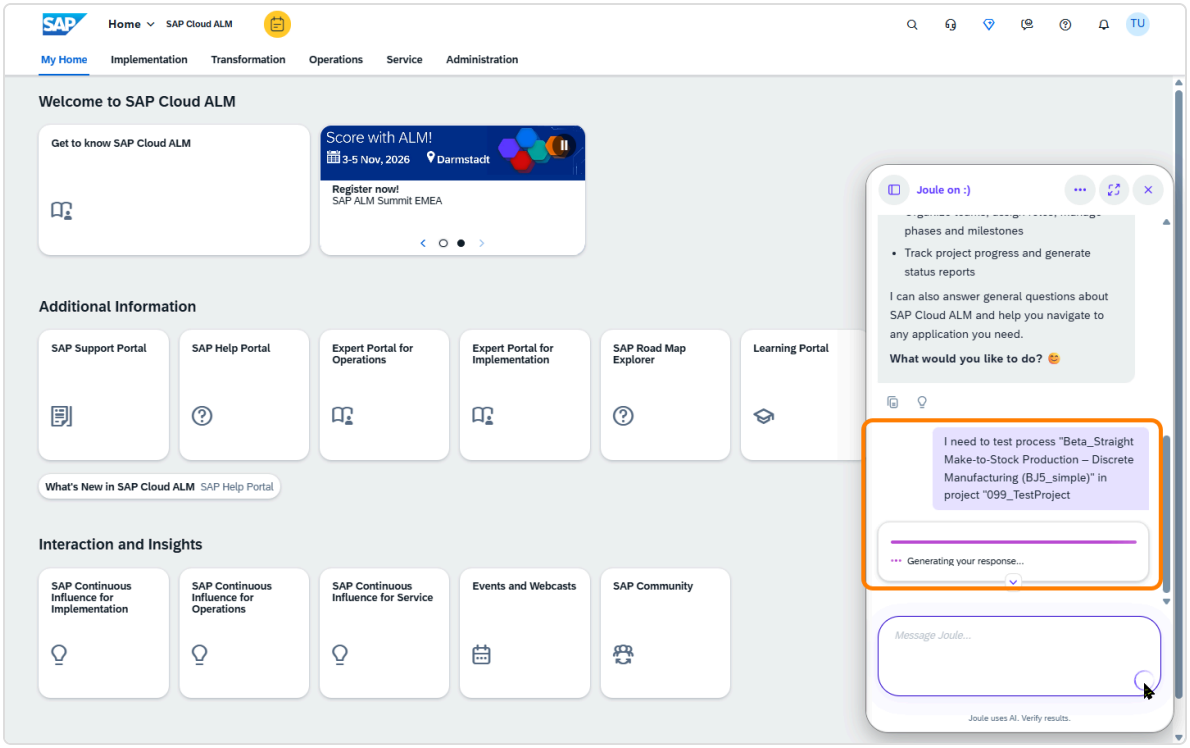
Time to try again; with proper input now in place.

3.1.1 – Prompt Joule (same as Step 1.3)

1. Open a new conversation in Joule and send the **same prompt** as in Step 1.3:

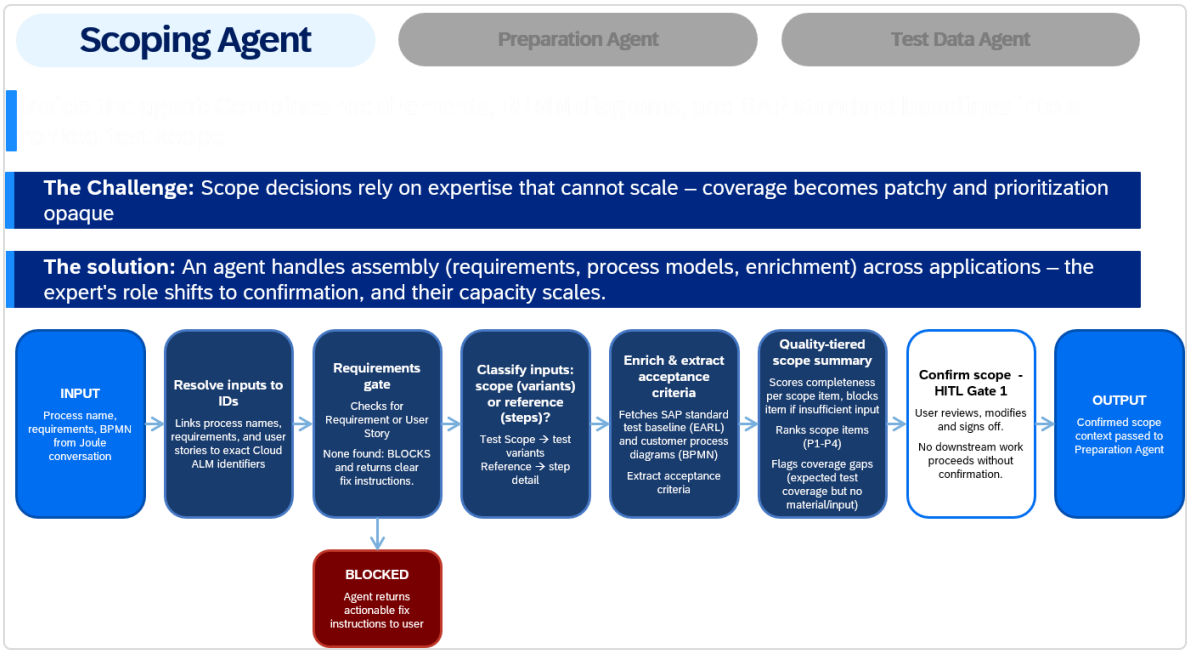
> *"I need to test process "Beta_Straight Make-to-Stock Production – Discrete Manufacturing (BJ5_simple)" in project "###_TestProject""*
>
> Replace **###** with your workstation number.

1. The scoping agent starts working in the background. You'll see a notification or progress indicator.



02 – Scoping agent running

3.2 – What's Happening? (Narration)



03 – Scoping agent methodology

> The scoping agent applies a structured methodology:
> 1. It resolves the process name, requirements, and user stories to exact identifiers in

> SAP Cloud ALM
> 2. It validates that the required input exists; if not, it blocks and returns clear instructions on what to fix
> 3. It classifies each input: process scope variants or detailed reference steps
> 4. It enriches the scope with SAP standard test baselines and extracts acceptance criteria from your linked content
> 5. It scores each scope item for completeness, ranks items by priority, and flags coverage gaps
> 6. It presents the result for your review and sign-off — no downstream work starts without your approval (HITL Gate 1)
>
> This is the same analysis a skilled test architect would do, but compressed from hours into minutes.

3.3 – Manual Scoping Exercise

While you wait for the agent, try a structured manual scoping exercise.

For the BJ5 process, work through these dimensions on a piece of paper or sticky note:

Dimension	Your Answer
What **acceptance criteria** can you identify from the user stories and requirements?	
How many distinct **apps/UIs** are involved?	
How many **roles** participate?	
What are the main **process paths** (happy path + variants)?	
Which steps have **blocking conditions** worth testing?	
What **error scenarios** should be covered?	

> Hint: The BJ5 process involves 2 roles (Production Planner and Warehouse Clerk) and spans the full make-to-stock cycle from production order creation to finished goods receipt. There is 1 critical quality gate where output is assessed before goods receipt — and a separate failure path for scrap and replenishment.

Keep your notes; you'll compare them to the agent's output next.

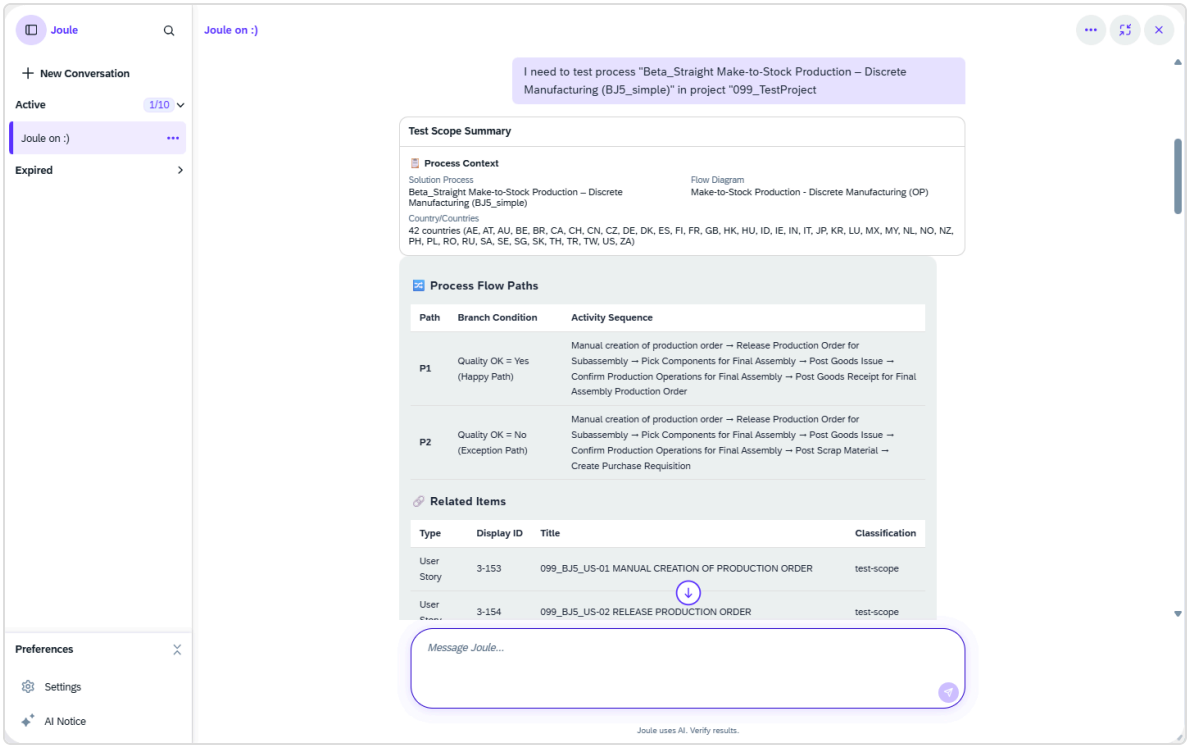
3.4 – Review the Scoping Agent's Work

3.4.1 – Open the Agent's Output

When the agent is ready, it replies directly in the Joule chat window with a **Test Scope Summary**. The response is detailed — switch Joule to **full-screen mode** first using the **expand icon** (⤢) in the top-right of the Joule panel for a much easier reading experience.

The response gives you:

- **Process context**: the solution process, flow diagram, and applicable countries it resolved
- A **summary of identified process flow paths** — reflecting any splits or variants in the process structure
- An overview of the **items it ingested**: your requirement and the 7 user stories
- Related **standard process procedures** retrieved from the SAP Enterprise Architecture Reference Library (EARL)
- A conclusive list of **acceptance criteria** derived from all of the above



04 – Test Scope Summary in Joule

3.4.2 – Evaluate and Compare

Take a few minutes to work through the proposed scope. For each item, check:
- **Name and description**: does it clearly describe what's being tested?

- **Linked acceptance criteria**: is it traceable back to a user story or requirement?
- **Role separation**: are Production Planner tasks and Warehouse Clerk tasks kept in separate test cases?
- **Coverage**: does the overall set cover the happy path (P1), the quality gate failure path (P2), and the scrap/replenishment scenario?

> **Compare with your manual notes from Step 3.3:** Did the agent catch everything you identified? Did it find things you missed? This is the key learning moment of this chapter.

When you're satisfied with the scope, signal your approval in Joule. This is **HITL Gate 1**: the preparation agent will not begin building the test plan until you confirm.

3.5 – (Optional) Refine the Scope in Joule

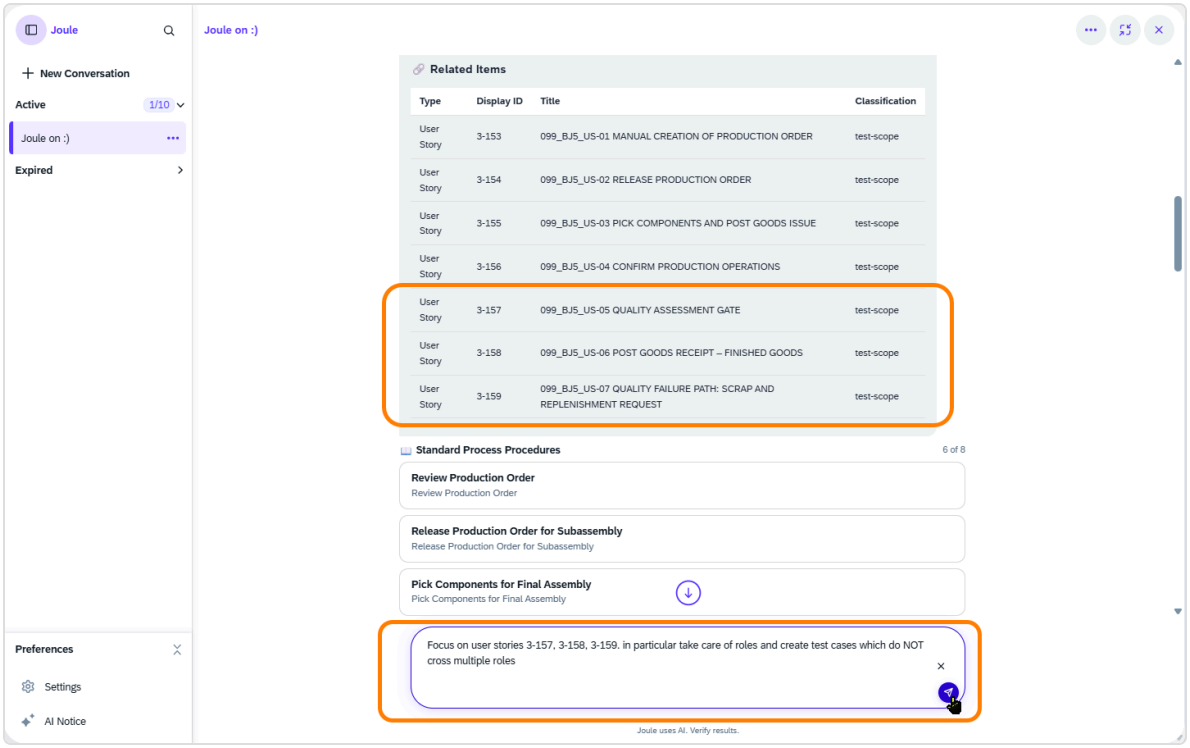
The scope is a starting point, not a final decision. Joule lets you refine it through conversation.

> **Note for participants sharing a system:** The agents run on shared infrastructure. To avoid overloading the system with concurrent calls, keep your refinement focused. We recommend limiting the scope to **user stories 5, 6, and 7** (the quality gate, goods receipt, and failure path).

Try this prompt as a starting point:

> *"Focus on user stories 3-157, 3-158, 3-159. In particular take care of roles and create test cases which do NOT cross multiple roles."*

Or try one of these on your own:
- **Narrow the scope:** "Remove user stories 1–4 from the scope for now; focus only on the quality gate, goods receipt, and scrap path."
- **Split a test case by role:** "Split the goods issue test case into two; one for the Production Planner and one for the Warehouse Clerk."
- **Add coverage:** "Add a negative test case for the quality gate failure path."



05 – Refining scope: focus on user stories 5–7

Summary

The scoping agent analyzed all your linked process artifacts and proposed a structured test case scope; with traceability to requirements and user stories; in the time it took you to sketch a rough outline manually.

You now have a **confirmed test case scope** ready for test planning.

Continue to Chapter 4 – Preparation Agent: Test Plan

Chapter 4 – Preparation Agent: Test Plan

Prerequisite: You have completed Chapter 3; a confirmed test case scope exists for the BKJ process.

What You'll Do

In this chapter you'll unleash the **Planning Agent**: an AI agent that takes your test case scope and builds a structured **test plan**: organized test case entries with descriptions, assigned roles, preconditions, and logical groupings. While it works, you'll compare it with the effort of doing this manually.

4.1 – Trigger the Planning Agent

4.1.1 – Confirm the Scope and Prompt for Planning

1. Make sure you're satisfied with the test case scope from Chapter 3. If you made refinements via Joule, confirm the scope is in its final state.
2. Open Joule and enter: **"The scope looks good. Please create a test plan."** or **"Confirm scope, go into planning."**



01 – Confirming scope and triggering the planning agent

1. The planning agent queues and starts. You'll see a running indicator.



02 – Planning agent running

> ⏱ **While the agent works: try it yourself:**
>
> Take one of the test cases from the scope and try to manually write a test plan entry:
>
> 1. Pick a test case (e.g. "Create Sales Order with Billing Plan")
> 2. Try to write: a clear description, the role performing it, the preconditions (what must be true before this step), and a rough sequence of test steps
> 3. How long does that take for one test case? Now multiply by the full scope...

This is what test plan preparation normally looks like.

4.2 – What's Happening? (Narration)

The challenge: Scope decisions rely on expertise that cannot scale; coverage becomes patchy and acceptance criteria unreliable. For each test case a test analyst would normally have to determine which app to use, define preconditions, sequence the steps at the right level of detail, assign the correct role, and link everything back to requirements. Multiply that by a full BKJ scope.

The solution: The **Preparation Agent** maps execution paths from the process model into complete test cases. The expert shifts from producing to reviewing, and covers scope more reliably.

Here is what the agent is doing right now, step by step:

Stage	What happens
Validate incoming scope	Checks quality and completeness of the scope context from the Scoping Agent. Flags issues before test generation begins.
Map all process variants	Reads the BPMN diagram to identify every execution path: happy path, error flows, boundary conditions, auth checks, negative flows.
Draft test plan	Creates a categorised plan covering all identified process variants; this is what you'll review and approve in Step 4.3.
HITL Gate 2: Approve plan	⛔ **No test cases are written to Cloud ALM without your explicit approval.** You review the plan and adjust before the agent continues.
Write test cases	Creates test cases in Cloud ALM via ContentAssist. Progress is streamed live so you can watch it work.
Link cases and apply EARL	Links each test case to its Requirement and Solution Process. Applies SAP standard content (EARL) where available.
Output	Complete, traceable test cases written to Cloud ALM Test Management; ready for execution planning.

The gate in the middle (HITL Gate 2) is deliberate: the agent cannot proceed past the plan without human sign-off. You are always in control.

4.3 – Review the Agent's Test Plan

4.3.1 – Open the Completed Plan

1. Go to the **Notification Area** and open the planning agent's completion notification.
2. Review the generated test plan; you should see test cases organized with descriptions, roles, and preliminary step structure.



03 – Test plan completion notification



04 – Generated test plan overview

4.3.2 – Inspect a Test Case

1. Open a test case in detail; look at the groupings, preconditions, and step-level descriptions.



05 – Test case detail: role, preconditions, steps

> **Traceability check:** Does the test case show its link back to the scope item and the user story? Good test plans maintain this chain; it's essential for coverage reporting and audit.

4.3.3 – Check Role Transitions

One of the most common planning mistakes is mixing multiple roles within a single test case. Look at the BKJ process; it involves 4 roles with 5 handover points.

> **Look for:** Are there test cases in the plan that should be split by role? For example, is the delivery step (Shipping Specialist) properly separated from the billing step (Billing Clerk)?



06 – Role boundary in a test case

4.4 – (Optional) Refine the Test Plan in Joule

Use Joule to make targeted adjustments to the plan without restarting from scratch.

Try one of these:

- **Split by role:** "Split test case 'Create and Release Billing Documents' into two; one for the Internal Sales Rep (billing block release) and one for the Billing Clerk (billing document creation)."
- **Add a precondition:** "Add a precondition to the delivery test case: down payment must be cleared before delivery creation can proceed."
- **Drop a step:** "Remove the stock setup step from the main test case; it's covered by a prerequisite setup script."



07 – Refining the test plan in Joule

Summary

The planning agent structured your test case scope into a workable test plan; with roles, preconditions, and step groupings; in minutes.

You now have a **confirmed test plan** ready for test case generation.

Continue to Chapter 5 – Preparation Agent: Test Cases

Chapter 5 – Preparation Agent: Test Cases

Prerequisite: You have completed Chapter 4; a confirmed test plan exists for the BKJ process.

What You'll Do

In this chapter you'll unleash the **Test Case Generation Agent**: an AI agent that takes the test plan and writes detailed, executable test cases. While it works, you'll draft one test case yourself, then compare results. After that you'll dive into one of the most overlooked aspects of test case quality: **test data and parameters**.

5.1 – Trigger the TC Generation Agent

5.1.1 – Confirm the Plan and Prompt for Generation

1. Make sure your test plan from Chapter 4 is in its final state.
2. Open Joule and enter: **"The plan looks good. Please generate the test cases."** or **"Confirm plan, start test creation."**



01 – Confirming plan and triggering test case generation

1. The generation agent starts. This is the most comprehensive agent; it writes detailed steps, expected results, and parameter placeholders for every test case in the plan.



02 – Test case generation agent running

> ⏱ **While the agent works: try it yourself:**
>
> Pick one test case from your plan (e.g. "Post Incoming Payment and Clear Down Payment") and try to write it manually:
>
> 1. Break it into functional units: what app? what action?
> 2. Assign the correct role: AR Accountant
> 3. Write the test steps at the right level of detail (not too high, not too granular)

4. For each step, write the expected result

In particular: note down any input parameters (company code, posting date, customer account, amount, house bank). Draw arrows to show dependencies; which parameter values are determined by earlier steps?

Keep your notes for Step 5.4.

5.2 – Manual Test Case Work

While the agent runs, try a more structured breakdown:

For your chosen test case, fill in this template:

Element	Your Draft
Test Case Name	
Role	AR Accountant
Precondition	
App	Post Incoming Payments (F1345)
Step 1	Open the app, select company code
Step 1: Expected Result	
Step 2	Enter posting date and journal entry type DZ
Step 2: Expected Result	
Step 3	Filter for Special G/L transactions, find FAZ open item
Step 3: Expected Result	
Step 4	Select FAZ item, choose Clear, Post
Step 4: Expected Result	
Parameters needed	Company code, posting date, G/L account, house bank, customer, amount

---

5.3 – Review the Generated Test Case

The agent has finished. Let's see what it produced.

5.3.1 – Open the First Test Case

1. Go to the **Notification Area** and open the TC generation agent's completion notification.



03 – Test case generation completion notification

1. Navigate to the first generated test case and open it in detail.



04 – Generated test case overview



05 – Test case detail: steps and expected results

5.3.2 – Quality Check

Review the generated test case against these criteria:

Quality Criterion	Check
Correct role assigned	☐
Clear, actionable steps (not too vague, not too granular)	☐
Expected result for every step	☐
Input parameters identified	☐
Traceable to a user story or requirement	☐
Logical grouping of steps	☐

> **Compare with your manual draft:** Did the agent's version match what you wrote? What did it do better? What would you change?

5.4 – Understand Test Data Parameters

This is where test preparation often gets underestimated. Let's go deeper.

5.4.1 – Identify the Parameters

Open the test case you've been looking at (the payment posting case, or any other complex one) and focus on the **input parameters**: the specific values that must be entered to execute each step.

1. For each step in the test case, note down all parameter fields (not the values yet, just the parameter names).



06 – Test case parameter fields

1. Now think about **dependencies** between parameters. For the BKJ process:
2. The **Sales Organization** (e.g. 1010) determines which **Distribution Channel** and **Division** are available
3. The **Sold-to Party** (customer) determines the **Ship-to Party** and **Payer**
4. The **Plant** (e.g. 1010) determines which **Storage Location** is available
5. The **Material** type determines which **Item Category** is auto-assigned (CBAO, CTAD, or CTAX)

Draw or describe these dependency relationships; which parameter determines which others?



07 – Parameter dependencies

> **This dependency map is what the Data Agent will use in Chapter 6**: it's not just about picking values, it's about picking consistent combinations.

5.4.2 – Note the Test Case ID

1. Note down the **test case ID** of the case you've been reviewing. You'll use it to prompt the data agent in Chapter 6.



08 – Test case ID for data agent

Summary

The TC generation agent wrote detailed, executable test cases for the full BKJ scope; complete with steps, expected results, and parameter placeholders.

You now understand:
- What a high-quality AI-generated test case looks like
- How to evaluate it for correctness, completeness, and traceability
- That **test data and parameters** are a critical dimension that needs its own attention

Continue to Chapter 6 – Test Data Agent

Chapter 6 – Test Data Agent

Prerequisite: You have completed Chapter 5; generated test cases exist and you have noted a test case ID.

What You'll Do

In this chapter you'll unleash the **Data Agent**: an AI agent that analyzes your test cases and proposes meaningful test data: parameter values with the right combinations, boundary cases, and reasoning for each choice. This is the final AI-assisted stage of test preparation.

6.1 – Trigger the Data Agent

6.1.1 – Prompt Joule with Your Test Case

1. Open Joule.
2. Enter a prompt that references the test case ID you noted in Step 5.4.2. Something like: **"Please suggest test data for test case [your-TC-ID]."**



01 – Triggering the data agent

1. The data agent queues and starts analyzing the test case; reading its steps, identifying parameters, and cross-referencing the master data requirements from the solution design.



02 – Data agent running

> ⏱ **While the agent works:**
>
> Think about what makes good test data:
> - Values that are **valid but exercise boundaries** (e.g. a down payment that is exactly 50% of the order value)
> - Values that are **invalid** but in realistic ways (e.g. insufficient stock, wrong billing type)
> - Values that **respect dependencies** (sales org determines customer pool, plant determines storage)

> What would you suggest as test data for the payment posting test case?

6.2 – What's Happening? (Narration)

> **The challenge:** Finding valid test data demands expert knowledge that doesn't scale; wrong data corrupts test results and steals time from actual testing.
>
> **The solution:** The **Test Data Management Agent** identifies every required data parameter from test case instructions; the tester looks up the known targets; no expert required. *(This is Phase 1 of a longer roadmap.)*
>
> Here is what the agent is working through right now:

Stage	What happens
Fetch test case	Reads the test case and all its activity steps from Cloud ALM via MCP (TL102). ⚠️ If the test case is not found, has no activities, or the text is empty; the agent stops and asks you to verify the input.
Extract parameters	Identifies every required SAP business object and field value from the instruction text; every line of "Enter Sales Organization..." becomes a structured parameter.
Classify parameters	Labels each parameter by category: **Org Data · Master Data · Config Data · Transactional Data**.
Apply project context	Obtains fixed values (company code, plant, client) from the Cloud ALM project record; so you don't have to look those up yourself.
Output	A read-only parameter table per category. Structured guidance for the tester who locates the actual system values manually.

> **Phase 1 scope note:** The agent identifies and classifies parameters; it does not yet automatically look up system values. That's the tester's final step: use the table as a checklist to find values in the target system.

6.3 – Review the Test Data Agent Output

6.3.1 – Open the Suggested Data

1. Go to the **Notification Area** and open the data agent's completion.



03 – Data agent completion notification

1. Review the suggested test data; you should see a structured set of parameter values with explanations.



04 – Test data suggestion: parameter table

6.3.2 – Evaluate the Data Quality

For each parameter group, check:

Aspect	What to look for
Completeness	Are all required parameters covered? Any missing?
Consistency	Are the values consistent with each other (e.g. correct sales org + customer combination)?
Coverage	Does the set include both valid (happy path) and invalid (negative test) values?
Reasoning	Does the agent explain *why* specific values were chosen?
Boundary cases	Are there values that test edge conditions (max amount, exact threshold)?



05 – Evaluating data quality and reasoning

6.4 – Go Deeper into Data Understanding

6.4.1 – Explore Dependencies

1. In Joule, ask: **"What are the dependencies between these parameters?"** or **"Which parameters drive the values of other parameters?"**

The agent should outline the dependency chain; for example, that the Sales Organization determines which customers and materials are valid.



06 – Parameter dependency chain

> **Key insight:** The dependency map the agent produces here is exactly what you started sketching in Step 5.4. Notice how parameters like Sales Organization (1010), Plant (1010), and Material type cascade into choices for all other fields.

6.4.2 – Reference Data in Context

For the BKJ process, the core master data values are:

Parameter	Value (Demo System)	Determines
Sales Organization	1010	Dist. Channel, Division, Customer pool
Distribution Channel	10 (Direct Sales)	Available customers
Division	00	–
Sold-to Party	10100004	Ship-to, Payer
Plant	1010	Storage location, available materials
Storage Location	101A	–
Material	TG11 / TG12	Item Category (CBAO)
Item Category	CBAO	Billing plan type
Shipping Point	1010	Delivery creation
House Bank	DEBK1 / DEAC1	Payment posting

> **Additional scenario:** The agent's real value shows when you have **unexpected combinations**: e.g. what happens if a non-stock material (NS0002) is used instead of a trading good? The item category changes to CTAX, which uses a different billing plan configuration. These scenarios are hard to spot manually but the agent flags them as meaningful test variants.



07 – Test data variants and edge cases

6.5 – Recap: Value Created

Let's step back and look at what you've accomplished in this hands-on.

Time comparison

Stage	Manual Effort (estimate)	AI-Assisted
Requirements from transcript	2–4 hours (BA writing)	~2 minutes (generation) + review
Test case scoping	4–8 hours (architect)	~3 minutes (agent) + review
Test plan creation	2–4 hours (test analyst)	~3 minutes (agent) + review
Test case writing	1–2 days (test analyst)	~5 minutes (agent) + review
Test data suggestion	4–8 hours (analyst + BA)	~2 minutes (agent) + review
Total	**3–5 days**	**< 30 minutes**



08 – Full test preparation cycle complete

> **The shift:** You didn't just save time; you changed the nature of the work. Instead of *creating* artifacts from scratch, you spent your time *reviewing and validating* AI-generated artifacts. That's the role of the future test expert: quality judgment, not manual production.

Summary

The data agent provided meaningful, dependency-aware test data suggestions with reasoning; turning a multi-hour analysis task into a review exercise.

You've now completed the full AI-assisted test preparation cycle:
1. ✅ Requirements generated from interview transcript
2. ✅ Test case scope proposed by scoping agent
3. ✅ Test plan structured by planning agent
4. ✅ Test cases written by TC generation agent
5. ✅ Test data suggested by data agent

Continue to Chapter 7 – Recap & Outlook

Chapter 7 – Recap & Outlook

Prerequisite: You have completed Chapters 1–6.

7.1 – Recap: A Day in the Life

You just experienced a complete AI-assisted test preparation workflow; from zero context to ready-to-execute test cases with test data; for a real, complex SAP S/4HANA business process.

Here's the journey you took:

```
Day starts → Process assigned (BKJ)
     │
     ├─ Requirements generated from expert interview transcript
     │     └─ No BA time, no writing; AI reads and structures
     │
     ├─ Scoping agent analyzes process artifacts
     │     └─ Produces test case scope with traceability
     │
     ├─ Planning agent structures the scope
     │     └─ Test plan with roles, preconditions, groupings
     │
     ├─ TC generation agent writes the test cases
     │     └─ Executable steps with expected results
     │
     └─ Data agent suggests test data
           └─ Parameter values, dependencies, variants
```

What used to take 3–5 days now takes < 30 minutes of AI-driven generation + your expert review.

The key shift: you moved from *producing* test artifacts to *judging* them. Your expertise is applied where it matters most; validating quality, not filling templates.

7.2 – Outlook: What's Coming Next

What You Saw Today Is Just the Beginning

The AI capabilities you worked with are the first wave. Here's where SAP Cloud ALM Test Management is heading:



01 – Product roadmap: upcoming capabilities

Wave 2: Coming enhancements:

Capability	What it means
Test execution assistance	AI guides the tester through steps in real time, compares system state to expected results
Defect auto-classification	AI analyzes failures and suggests root cause categories
Coverage gap detection	AI identifies process areas with insufficient test coverage
Cross-scope impact analysis	AI flags when a scope item change affects existing test cases
EARL knowledge integration	Deeper use of SAP Best Practice content (EARL) for more precise step generation

> **The bigger picture:** SAP Cloud ALM is building toward a **360° test expert role**: someone who orchestrates AI agents across the full test lifecycle rather than manually executing each stage. Today you got a preview of that future.

7.3 – This Is a Beginning, Not the End

Key Takeaways

1. **Input quality drives output quality.** The agents worked well because you provided complete, high-quality input: requirements, user stories, solution description. Garbage in, garbage out; still applies to AI.
2. **Human judgment is the irreplaceable layer.** The agents produced the first draft. You provided the quality review; catching role mismatches, missing preconditions, incorrect scope. That's not going away.
3. **The workflow is the product.** It's not about any single AI feature. It's about the **end-to-end pipeline** from process documentation to executable test cases; with AI at every stage.
4. **Traceability is preserved.** Every generated artifact is linked back through the chain: test data → test case → test plan → scope → user story → requirements → process. This is what makes AI-generated content auditable and trustworthy.

Questions & Discussion

▶ 📷 View session closing screenshot

Some questions to discuss:

1. Where in your current testing process would AI agents save you the most time?
2. What inputs do you have today that you'd want to feed to a scoping agent?
3. What does "human in the loop" mean for your team; at which stage would you want to review and confirm?

Feedback

We'd love to hear from you:
- [Leave feedback via the QR code at the front of the room]
- [Or at the session feedback survey link]

Thank You

TM261: AI-Powered Test Management with SAP Cloud ALM

This material will remain available after the session at the SAP TechEd GitHub repository.